



Treball Final de Grau
Grau en Matemàtiques

**Modelització i optimització
de trajectes en la xarxa
de transport públic de Barcelona**

NIU: 1598205

Curs 2025–2026

Resum

Aquest treball estudia el problema del camí mínim entre dos accessos de la xarxa de metro de Barcelona. A partir de les dades oficials GTFS de TMB, es construeix un graf dirigit ponderat que modela aquesta infraestructura, incloent-hi el Funicular de Montjuïc, sobre el qual s'apliquen algoritmes de camí mínim.

A més de la construcció del model, s'analitzen diferents estratègies per reduir el nombre d'iteracions necessàries per trobar el recorregut òptim. En particular, es desenvolupa una heurística específica basada en la topologia de la xarxa i una tècnica de reducció del graf de treball, que es comparen amb Dijkstra i amb A* utilitzant una heurística geomètrica convencional.

Els resultats mostren que la combinació d'un model adequat, d'una heurística adaptada al problema i d'una tècnica de reducció del graf permet millorar significativament l'eficiència de la cerca, assolint una reducció aproximada d'un factor de 6.5 respecte de la solució inicial basada en Dijkstra.

Abstract

This thesis studies the shortest path problem between two access points of the Barcelona subway network. Using the official GTFS data provided by TMB, a weighted directed graph is constructed to model the network, including the Montjuïc Funicular, on which shortest path algorithms are applied.

In addition to the graph construction process, several strategies are investigated to reduce the number of iterations required to find the optimal route. In particular, a network-specific heuristic based on the topology of the Barcelona metro system and a graph reduction technique are developed and compared with Dijkstra's algorithm and A* using a conventional geometric heuristic.

The results show that combining an appropriate graph model, a problem-specific heuristic, and a graph reduction technique significantly improves search efficiency, achieving an approximately 6.5-fold reduction in the number of iterations required compared with the initial solution based on Dijkstra's algorithm.

M'agradaria expressar el meu agraïment al meu tutor/a per la seva guia i el seu suport al llarg de la realització d'aquest treball. També al Lluís Alsedà pels seus comentaris i les seves valuoses indicacions.

I, en especial, voldria agrair a la meua família i a les meves amistats, que han confiat en mi i m'han encoratjat a continuar endavant durant tots aquests anys.

Índex

1	Introducció	1
2	Marc teòric	3
2.1	Camins mínims a grafs ponderats	3
2.2	Algoritmes de camí mínim	5
2.2.1	L'algoritme Dijkstra	5
2.2.2	L'algoritme A*	6
3	Construcció del graf	8
3.1	Introducció	8
3.2	Definició dels pesos	9
3.3	Model del graf	10
3.3.1	Decisions del model del graf	10
3.3.2	Introducció al cas particular de la parada L3 – Liceu	11
3.4	Assignació dels pesos SW	11
3.5	Graf resultant	12
4	Cerca de camins mínims	15
4.1	Introducció	15
4.2	Dijkstra	15
4.3	Rendiment dels algoritmes	16
4.4	A _{cheat} *	16
4.5	Metodologia d'avaluació	17
4.6	A _{geo} *	18
4.6.1	Definició de l'heurística h_{geo}	18
4.6.2	Avaluació experimental	20
4.7	Topologia particular del graf	20
4.7.1	Motivació	20
4.7.2	Definició de les regions	21
4.7.3	Casos segons les regions	22
4.7.4	Validació experimental i anàlisi	22
4.8	A _{bcn} *	26
4.8.1	Definició de l'heurística h_{bcn}	26
4.8.2	Avaluació experimental i anàlisi	30
4.9	Graf de treball reduït	32
4.9.1	Motivació i construcció	32
4.9.2	Avaluació experimental i anàlisi	33

5	Conclusions	34
	Referències	37
A	Pseudocodis d'algoritmes	38
B	Construcció del graf	40
B.1	Pipeline de validació i processament de dades	40
B.1.1	Etapas del pipeline principal	40
B.1.2	El bucle de detecció, correcció i verificació	44
B.1.3	Comprovacions independents	46
B.1.4	Anàlisis de suport a les decisions de disseny	49
B.1.5	Consideracions finals	49
B.2	Sortides de programes	49
C	Cerca de camins mínims	51
C.1	Cas particular de la parada L3 – Liceu	51
C.2	Exemple d'una proporció inferior a 1 amb l'heurística ideal	52
C.3	Exemples de nodes explorats	52
D	Andanes i entrades del graf	60

Índex de figures

1	Xarxa de metro de Barcelona	2
2	Graf construït	14
3	Regions i ponts del graf	23
4	Rendiment mitjà segons el cas amb A^*_{bcn}	31
5	Rendiment mitjà segons el cas amb el graf reduït	34
6	Etapes del pipeline principal	41
7	Patró de detecció, correcció i verificació	45
8	Entrada Alcolea (escala mecànica) de L1–Plaça de Sants	47
9	Exemple cas BC	53
10	Exemple cas CB	54
11	Exemple cas CC amb A^*_{bcn}	55
12	Primer exemple cas DB amb A^*_{bcn}	56
13	Segon exemple cas DB amb A^*_{bcn}	57
14	Mateix exemple cas CB però amb A^*_{bcn}	58
15	Mateix exemple cas CC però amb el graf de treball reduït	59

Índex de taules

1	Atributs rellevants dels fitxers GTFS utilitzats	8
2	Andanes compartides entre les línies L9 i L10	10
3	Característiques del graf construït	13
4	Definició de les branques del graf	21
5	Pont associat a cada branca	22
6	Rendiment mitjà segons el cas	24
7	Temps de portes	43
8	Connexions per andanes sense entrades	48
9	Aparellament de <code>trip_headsign</code> i <code>direction_id</code> per línia	48

1 Introducció

La mobilitat dins les grans ciutats és un problema d'interès tant social com científic. Cada dia, milers de persones utilitzen aplicacions com Google Maps per trobar el recorregut més ràpid o més convenient entre dos punts de la ciutat. Darrere d'aquestes eines hi ha problemes clàssics d'optimització sobre grafs, en què cal determinar un camí òptim entre un origen i un destí en el menor cost computacional possible.

Aquest treball es centra en el cas particular de la xarxa de metro de Barcelona, incloent-hi la línia del Funicular de Montjuïc. A partir de dades oficials en format General Transit Feed Specification (GTFS) es construeix un model en forma de graf que representa la xarxa i permet estudiar el problema del camí mínim entre un origen i un destí aplicat a dos accessos de la xarxa. L'objectiu no és únicament trobar el recorregut de temps mínim entre dos punts, sinó també reduir el nombre d'iteracions necessàries per obtenir-lo (vegeu la definició d'iteració emprada a la Secció 4.1).

Per assolir aquest objectiu es parteix de l'algoritme Dijkstra com a referència i es desenvolupen dues heurístiques específiques per a l'algoritme A^* , a més d'una heurística de referència utilitzada com a límit superior del rendiment assolible. Cada nova proposta sorgeix de les limitacions observades en l'anterior. Aquest procés culmina amb el desenvolupament d'una heurística específica per al metro de Barcelona i, posteriorment, amb una modificació del graf de treball que encara permet reduir més el nombre d'iteracions.

La narrativa del treball segueix, sempre que és possible, el mateix procés de raonament que ha conduït als resultats presentats. D'aquesta manera, cada nova idea apareix motivada per les limitacions observades en les anteriors, reproduint el procés que ha portat fins a la proposta final.

L'estudi es basa en diverses hipòtesis de modelització derivades tant de les dades disponibles com de l'abast del treball. Entre les més rellevants destaquen que es treballa amb dades del setembre de 2025, que el temps es mesura amb precisió d'un segon, que els recorreguts són independents del moment del dia i que es considera un sistema ideal en què no existeixen temps d'espera dels trens ni limitacions derivades de l'afluència d'usuaris. Aquestes i la resta d'hipòtesis es descriuen detalladament durant la construcció del model.

Cal destacar que les aportacions pròpies d'aquest treball no es limiten a la implementació dels algoritmes descrits. A partir del marc teòric presentat, s'ha desenvolupat íntegrament la construcció del model del graf a partir de dades GTFS, s'han formulat les hipòtesis i simplificacions necessàries per adaptar el problema al cas d'estudi, s'han dissenyat dues noves heurístiques (una general i l'altra específica per a la xarxa amb la què es treballa), s'ha proposat una nova estratègia basada en un graf de treball reduït i s'ha dut a terme tota l'avaluació experimental dels algoritmes.

A més, els exemples, les figures i les anàlisis experimentals que apareixen al llarg de la memòria són originals, excepte quan s'indica explícitament el contrari.

Com a referència, s'inclouen el mapa oficial de la xarxa de metro de TMB (Figura 1), el llistat complet de parades amb les seves entrades a `stops_report.txt`, una extracció d'aquest al Llistat 4 i el repositori de codi de GitHub que dona suport a tota la part pràctica del treball.

La memòria s'organitza de la manera següent. En primer lloc es presenta el marc teòric ne-



Figura 1: Xarxa de metro de Barcelona. Font: Transports Metropolitans de Barcelona (TMB) [7]

cessari per al desenvolupament del treball (Secció 2). A continuació es descriu la construcció del graf a partir de les dades GTFS i les decisions de modelització adoptades (Secció 3). Posteriorment s'estudien diferents estratègies per resoldre el problema dels camins mínims, començant pel Dijkstra i evolucionant progressivament cap a heurístiques específiques i modificacions del graf de treball (Secció 4). Finalment, es presenten les conclusions obtingudes i es discuteixen possibles línies de treball futur (Secció 5).

Finalment, els annexos inclouen els pseudocodis dels algorismes, la documentació tècnica del pipeline de construcció del graf i diferent material complementari que dona suport als resultats presentats, incloent-hi sortides dels programes i exemples de nodes explorats en la cerca de camins mínims.

2 Marc teòric

El contingut d'aquesta secció es basa íntegrament en [1].

2.1 Camins mínims a grafs ponderats

Definició 2.1 (Graf). Un graf (combinatori) és un parell

$$G = (V, E),$$

on V és un conjunt de vèrtexs (o nodes) i

$$E \subseteq V \times V$$

és un subconjunt del producte cartesià $V \times V$.

Definició 2.2 (Graf no dirigit). Un graf és no dirigit si les arestes no tenen orientació. És a dir, si $(a, b) \in E$, aleshores (b, a) representa la mateixa aresta. En aquest cas, els elements d' E s'anomenen arestes.

Definició 2.3 (Graf dirigit). Un graf és dirigit si les arestes tenen orientació. En aquest cas, els elements d' E s'anomenen arcs, i un arc de a a b existeix si i només si

$$(a, b) \in E.$$

Els parells (a, b) i (b, a) es consideren diferents.

Definició 2.4 (Ordre d'un graf). L'ordre d'un graf és el nombre de vèrtexs que conté, és a dir, el cardinal del conjunt V : $|V|$.

Definició 2.5 (Mida d'un graf). La mida d'un graf és el nombre d'arestes (o d'arcs, si el graf és dirigit), és a dir, el cardinal del conjunt E : $|E|$.

Definició 2.6 (Camí). Un camí és una successió de vèrtexs

$$\alpha = v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_n,$$

tal que cada parell consecutiu de vèrtexs està unit per una aresta (o un arc). En un graf dirigit, l'orientació dels arcs ha de coincidir amb el sentit del recorregut.

Definició 2.7 (Graf ponderat). Un graf ponderat (o xarxa) és un graf en què a cada aresta (o arc, si el graf és dirigit) se li associa un nombre real, anomenat pes. Aquest pes pot representar, per exemple, un cost, una distància, un temps o una capacitat.

Definició 2.8 (Funció de pesos). Els pesos d'un graf es poden representar mitjançant una funció

$$\omega : E \longrightarrow \mathbb{R},$$

que associa a cada aresta (o arc) $e \in E$ el seu pes $\omega(e)$.

Definició 2.9 (Pes d'un camí). Sigui

$$\alpha = v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_n$$

un camí en un graf ponderat. El seu pes es defineix com

$$\omega(\alpha) = \sum_{i=1}^n \omega(v_{i-1}, v_i).$$

Definició 2.10 (Pes mínim). El pes mínim entre dos vèrtexs u i v es defineix com

$$\sigma(u, v) = \min\{\omega(\alpha) : \alpha \text{ és un camí de } u \text{ a } v\}.$$

Per convenció, si no existeix cap camí entre u i v , aleshores

$$\sigma(u, v) = \infty.$$

Observació 2.1. El pes mínim entre dos vèrtexs pot no existir, però, quan existeix, és únic.

Definició 2.11 (Camí mínim). Un camí mínim entre dos vèrtexs u i v és qualsevol camí de u a v amb pes igual a $\sigma(u, v)$, sempre que aquest pes existeixi.

Observació 2.2. Entre dos vèrtexs poden existir diversos camins mínims.

Definició 2.12 (Graf no negatiu). Un graf ponderat $G = (V, E, \omega)$ és no negatiu si

$$\omega(e) \geq 0, \quad \forall e \in E.$$

Definició 2.13 (Graf positiu). Un graf ponderat $G = (V, E, \omega)$ és positiu si

$$\omega(e) > 0, \quad \forall e \in E.$$

Definició 2.14 (Graf estrictament positiu). Un graf ponderat $G = (V, E, \omega)$ és estrictament positiu si existeix una constant $\tau > 0$ tal que

$$\omega(e) \geq \tau, \quad \forall e \in E.$$

Observació 2.3. Tot graf ponderat positiu i finit és també estrictament positiu.

Definició 2.15 (Problema del camí mínim des d'un únic origen). Sigui $G = (V, E, \omega)$ un graf ponderat estrictament positiu. Donat un vèrtex origen $\xi \in V$, el problema consisteix a trobar, per a cada vèrtex $v \in V$, un camí mínim des de ξ fins a v , així com el seu pes mínim $\sigma(\xi, v)$.

Observació 2.4. Les nocions de pes mínim i camí mínim només estan definides per a grafs ponderats no negatius. Tanmateix, per assegurar la convergència dels algorismes per al problema del camí mínim des d'un únic origen, es requerirà que el graf sigui estrictament positiu.

Definició 2.16 (Problema del camí mínim entre un origen i un destí). Sigui $G = (V, E, \omega)$ un graf ponderat estrictament positiu. Donats un vèrtex origen $\xi \in V$ i un vèrtex destí $\gamma \in V$, el problema consisteix a trobar un camí mínim des de ξ fins a γ , així com el seu pes mínim $\sigma(\xi, \gamma)$.

Observació 2.5. El problema del camí mínim entre un origen i un destí és un cas particular del problema del camí mínim des d'un únic origen.

Notació 2.1. A vegades s'abúsarà del llenguatge i en comptes d'escriure vèrtex o node d'origen o de destí, es referirà a ells com *Source* i *Target*.

Notació 2.2. Sovint, utilitzarem ξ i γ per referir-nos al node d'origen i de destí, respectivament.

2.2 Algoritmes de camí mínim

En aquesta secció estudiarem els algoritmes Dijkstra i A^* per resoldre els problemes de camí mínim.

Definició 2.17 (Algoritme admissible). Diem que un algoritme és admissible si garanteix retornar una solució òptima, sempre i quan aquesta existeixi.

2.2.1 L'algoritme Dijkstra

L'algoritme Dijkstra resol el problema del camí mínim des d'un únic origen en grafs ponderats. A partir d'un node origen, calcula de manera iterativa el cost mínim per arribar a la resta de vèrtexs accessibles. Per fer-ho, utilitza una estratègia *greedy*. És a dir, a cada iteració pren el vèrtex amb menor cost des de l'origen.

El pseudocodi es pot trobar a l'Algoritme 1.

Observació 2.6. En finalitzar l'algoritme, és possible que existeixin vèrtexs $v \in V$ tals que $\text{dist}[v] = \infty$. Això succeeix quan no existeix cap camí des del node origen fins a v . En un graf dirigit, aquest fet pot produir-se encara que el graf sigui connex com a graf no dirigit, simplement perquè no hi ha cap camí orientat que connecti l'origen amb aquell vèrtex.

Teorema 2.1 (Convergència de l'algoritme de Dijkstra). La igualtat

$$\text{dist}[v] = \sigma(\xi, v)$$

es compleix sempre que un vèrtex $v \in V$ és extret de la cua de prioritat (mitjançant la funció `extract_min`) i expandit, i es manté certa durant la resta de l'execució de l'algoritme.

En particular, en finalitzar l'algoritme es té

$$\text{dist}[v] = \sigma(\xi, v)$$

per a tot vèrtex $v \in V$.

Observació 2.7. L'algoritme Dijkstra només depèn del graf i del node origen i, per tant, és independent del node de destí. Si només es vol obtenir el camí mínim fins a un únic destí, l'algoritme es pot aturar tan bon punt aquest vèrtex és extret de la cua de prioritat.

Observació 2.8. En finalitzar l'algoritme, tots els vèrtexs accessibles des de l'origen han estat expandits. Els vèrtexs que no ho han estat corresponen precisament als que no són accessibles, i mantenen el valor $\text{dist}[v] = \infty$.

Observació 2.9. El vector **parent** permet reconstruir qualsevol camí mínim calculat. En concret, per obtenir el camí fins a un vèrtex només cal seguir iterativament els seus predecessors fins arribar al node d'origen.

Observació 2.10. Com a conseqüència del teorema anterior, l'algoritme Dijkstra és admissible, ja que retorna un camí mínim des del node d'origen fins a qualsevol node accessible del graf.

2.2.2 L'algoritme A*

L'algoritme A* resol el problema del camí mínim entre un node d'origen i un node de destí en un graf ponderat. A diferència de l'algoritme de Dijkstra, que només té en compte la distància recorreguda des de l'origen, l'A* també utilitza una estimació del cost que encara resta fins al destí.

Definició 2.18 (Funció heurística). Sigui $G = (V, E, \omega)$ un graf ponderat. Una funció heurística és una aplicació

$$h : V \times V \longrightarrow \mathbb{R}_{\geq 0}$$

que associa a cada parell de nodes (v, γ) una estimació del cost restant necessari per arribar des de v fins al node de destí γ .

Observació 2.11. La funció heurística pot interpretar-se com l'error que s'assumeix sobre el cost restant fins al node de destí.

En cada iteració, l'algoritme selecciona el node que minimitza

$$f(v) := g(v) + h(v, \gamma),$$

on $g(v)$ és el cost del camí recorregut fins a v des de l'origen.

D'aquesta manera, A* combina el cost ja recorregut amb una estimació del cost restant, orientant la cerca cap al node de destí.

El pseudocodi es pot trobar a l'Algoritme 2.

Observació 2.12. L'algoritme A* depèn del graf, el node d'origen i el node de destí.

Definició 2.19 (Funció heurística admissible). Diem que una funció heurística h és admissible si $\forall v, \gamma \in V$, es té que

$$h(v, \gamma) \leq \sigma(v, \gamma).$$

Observació 2.13. Si h és una funció heurística admissible, aleshores $\forall v \in V$

$$h(v, v) = 0.$$

En efecte, com que $\sigma(v, v) = 0$ i h pren valors no negatius, necessàriament

$$0 \leq h(v, v) \leq \sigma(v, v) = 0.$$

Teorema 2.2 (Admissibilitat de l'algoritme A^*). Si la funció heurística és admissible, llavors l'algoritme A^* també ho és.

Observació 2.14. Com a conseqüència, l'algoritme A^* retorna un camí mínim sempre que la funció heurística sigui admissible.

Observació 2.15. L'algoritme A^* coincideix amb l'algoritme Dijkstra quan la funció heurística és idènticament nul·la.

Notació 2.3. Denotarem l'heurística nul·la per h_0 .

Observació 2.16. L'observació anterior posa de manifest la diferència essencial entre el Dijkstra i l' A^* . Amb h_0 , l' A^* selecciona els vèrtexs únicament segons el cost acumulat $g(v)$, i per tant coincideix amb el Dijkstra. En canvi, quan h proporciona informació sobre el cost restant, l' A^* continua sent un algoritme greedy, però ara respecte de $f(v)$, fet que orienta la cerca cap al node de destí i acostuma a reduir el nombre de vèrtexs explorats.

Definició 2.20 (Funció heurística consistent). Diem que una funció heurística h és consistent si $\forall u, v, \gamma \in V$, es compleix

$$h(u, \gamma) \leq \sigma(u, v) + h(v, \gamma).$$

Teorema 2.3. Tota funció heurística consistent és admissible.

Observació 2.17. A diferència de l'algoritme Dijkstra, l' A^* no garanteix, en general, que un vèrtex extret de la cua de prioritat no torni a ser inserit posteriorment. Això és degut al fet que la prioritat dels vèrtexs depèn de $f(v)$, i no únicament del cost acumulat $g(v)$.

Observació 2.18. Quan la funció heurística és consistent, el valor d' f és no decreixent al llarg dels camins explorats. En conseqüència, una vegada un vèrtex és extret de la cua de prioritat, el seu valor $g(v)$ ja és òptim i no serà necessari tornar-lo a inserir. Per tant, en aquest cas, l' A^* comparteix amb el Dijkstra la propietat que cada vèrtex s'expandeix com a màxim una vegada.

Observació 2.19. El pseudocodi de l'Algoritme 2 contempla el cas general en què la funció heurística només és admissible. En aquesta situació pot ser necessari tornar a inserir un vèrtex a la cua de prioritat si posteriorment es descobreix un camí de menor cost. Tot i que les heurístiques consistents eviten aquest comportament, verificar la consistència d'una heurística acostuma a ser complicat en aplicacions reals, ja que implica comprovar una propietat relacionada amb els costos òptims del graf, que precisament són desconeguts abans d'executar l'algoritme.

3 Construcció del graf

3.1 Introducció

L'objectiu d'aquest apartat és explicar com, a partir d'unes dades oficials i públiques, s'ha construït un graf dirigit ponderat estrictament positiu per poder aplicar els algoritmes que resolen el problema del camí mínim des d'un únic origen. En particular, es vol obtenir un graf en què els pesos de les arestes siguin constants, per simplicitat, i representin el temps necessari per viatjar d'un vèrtex a un altre. Això implica que aquests pesos no depenen de l'hora del dia ni del dia de la setmana en què s'inicia el recorregut.

Per a la construcció del graf s'han utilitzat dades en format General Transit Feed Specification (GTFS), un estàndard obert per a la representació d'informació sobre xarxes de transport públic, com ara parades, recorreguts, horaris i connexions [4]. En aquest treball s'han emprat les dades corresponents a l'empresa Transports Metropolitans de Barcelona (TMB), obtingudes a través del Punt Nacional d'Accés a les Dades del Transport (NAP) [3] i amb data de setembre de 2025.

La Taula 1 recull els fitxers i atributs de format GTFS que s'han utilitzat en aquest treball. Per a una descripció completa de cada fitxer i del significat dels seus atributs, es convida al lector a llegir la documentació oficial de GTFS a [4].

Taula 1: Atributs rellevants dels fitxers GTFS utilitzats

Fitxer	Columnes rellevants
pathways	Pathway_id, From_stop_id, To_stop_id, Traversal_time
routes	Route_id, Route_short_name, Route_long_name
stop_times	Trip_id, Arrival_time, Departure_time, Stop_id, Stop_sequence
stops	Stop_id, Stop_name, Stop_lat, Stop_long
transfers	From_stop_id, To_stop_id, Min_transfer_time
trips	Route_id, Trip_id, Trip_headsign, Direction_id

Les cel·les amb el mateix color indiquen els atributs que estableixen relacions entre els diferents fitxers.

Les comprovacions realitzades de l'estat i la qualitat de les dades han posat de manifest que, tot i que els fitxers GTFS contenen dades del metro, l'autobús i el funicular de Montjuïc, la informació disponible no és suficient per modelar correctament la xarxa d'autobús. En particular, el fitxer **transfers** no inclou les correspondències entre parades, fet que obligaria a incorporar informació addicional de la xarxa viària. Altres fitxers com **stop_times**, també presenten limitacions, com camps inexistents o inconsistents. Com que tractar aquests aspectes ampliaria considerablement l'abast del problema, en aquest treball només es consideren els mitjans de transport del metro i el funicular.

L'únic tram en el què no són explícits els dos sentits del recorregut és a l'entrada Alcolea (escala mecànica) de la parada L1 – Plaça de Sants (vegeu el Llistat 4). Oficialment, només es pot anar de l'andana cap a l'exterior i no a l'inrevés.

Tota aquesta secció es recolza en el directori **data_validation** del repositori. La construcció del graf no consisteix únicament en els processos que es descriuen a continuació, sinó que

ha requerit un procés previ de validació i preparació de les dades considerablement més ampli. En concret, el pipeline incorpora sis etapes de transformació, tres bucles de detecció, correcció i verificació, quatre scripts de comprovació amb dotze validacions independents i tres anàlisis empíriques que justifiquen les principals decisions de modelatge. Aquest procés es descriu amb detall a l'Annex B.1.

3.2 Definició dels pesos

Al graf tenim dos tipus de vèrtexs: entrades i andanes. A partir d'aquí podem definir els tipus de pesos segons les arestes i els vèrtexs.

D'entrada podríem pensar que hi ha quatre tipus diferents:

1. El temps que es camina
 - (a) d'una entrada a una andana (o a l'inrevès), o
 - (b) d'una andana a una andana (transbord)
2. El temps que el metro té les portes tancades i està en marxa
3. El temps que en una andana el metro té les portes obertes perquè pugi i baixi gent
4. El temps que passa quan s'espera a que arribi el metro

Simplificant-t'ho, obtenim la següent categorització de pesos.

- **Pathway (PW)**: el temps que es camina d'una entrada (d'origen o de destí) a una andana i viceversa.
És l'1(a), ve donat pel fitxer `pathways` i sempre és 60s (vist a l'Annex B.1.3).
- **Transfer (TF)**: el temps que camines d'una andana a una altra andana de la mateixa parada (transbord entre línies).
És l'1(b) i ve pel fitxer `transfers`.
- **Subway (SW)**: el temps entre que arriba el metro a una parada i arriba a la següent.
Inclou 2 i 3 i ho hem de calcular nosaltres.

S'ha inclòs 3 dins de SW perquè encara que els algorismes que farem servir permeten que un vèrtex tingui una aresta sortint i acabant en ell, aquest pes mai s'inclourà al pes mínim d'un camí. Com aquest tipus de pes sempre està present en una ruta, incloure'l aquí ha sigut la manera més senzilla que s'ha trobat per tal d'assegurar-nos que els algorismes el consideraran en el temps final.

Per simplicitat, considerarem 4 nul. És a dir, considerarem un sistema de transport ideal en el què cada vegada que l'usuari arriba a una andana no ha d'esperar a que arribi el metro per pujar-s'hi.

Per consistència, com els pesos PW i TF venen donats en segons (nombres enters), seguirem el mateix patró a tot el treball tant pels pesos SW com pels valors retornats per les funcions heurístiques.

3.3 Model del graf

A partir d'aquest punt, l'objectiu és completar el model del graf. Com que els pesos PW i TF es determinen directament a partir de les dades GTFS, abans d'assignar els pesos SW cal prendre diverses decisions estructurals que condicionen la representació de la xarxa. A més, convé analitzar un cas particular que no es pot representar completament amb el model adoptat.

3.3.1 Decisions del model del graf

En particular, cal decidir si el graf s'ha de modelar com a dirigit i com representar els trams compartits de les línies L9 i L10 (vegeu la Taula 2).

Taula 2: Andanes compartides entre les línies L9 i L10

Tram	Stop ID	Nom
Sud	1.914	Can Tries Gornal
	1.915	Torrassa
	1.916	Collblanc
Nord	1.930	La Sagrera
	1.932	Onze de Setembre
	1.933	Bon Pastor

Pel que fa a la direccionalitat, podria semblar suficient considerar un graf no dirigit, ja que l'única aresta no bidireccional és de tipus PW. Tanmateix, aquesta simplificació obligaria a introduir un tractament especial per a aquest cas, mentre que un graf dirigit permet representar de manera natural totes les connexions presents a les dades.

Per justificar aquestes decisions ens basarem en els resultats obtinguts amb els fitxers `directional_asymmetry.py` i `shared_platforms.py` del directori `data_validation/analysis`. Els resultats d'ambdós estudis tenen resultats complementaris. Per aquest motiu, es presenten conjuntament.

En tots dos casos, per a cada parell d'andanes consecutives d'una mateixa línia i per a cada sentit de circulació, s'ha calculat la mitjana del temps dels viatges presents al fitxer `stop_times`. A partir d'aquestes dades s'ha analitzat, d'una banda, l'efecte de la direcció del recorregut i, de l'altra, la necessitat de distingir els trams compartits de les línies L9 i L10.

Respecte la primera qüestió, per les 5 parelles d'andanes amb més diferència de temps segons la direcció del trajecte hem obtingut un desajust de més de 30s. Indicant, clarament, que necessitem un graf dirigit per no silenciar aquestes tendències al model. Aquestes discordances poden deure's a diferents profunditats en les què es troben les parades o altres factors que desconexim.

S'ha de recalcar que tot i tenir un graf dirigit i, per tant, arcs, si la direcció no és rellevant, sovint s'abusarà del llenguatge i es parlarà d'arestes.

Respecte la segona qüestió, només tenim un cas en el que el desajust arriba a més d'un segon, i per tant, l'únic rellevant per nosaltres. Concretament la diferència és de 2.82s. Indicant, doncs, que la duplicació d'aquestes andanes és convenient.

Per portar això a terme s'han de fer alguns canvis en els fitxers GTFS amb els què es treballen. Entre ells es troba l'addició de pesos TF per connectar les andanes sintètiques entre elles. S'ha decidit que aquests tinguin valor de 15 s ja que és el temps mínim present a **transfers** i és el que s'estima en un mitjà de transport ideal que es triga en baixar d'un metro d'una línia, que aquest se'n vagi justament després, que arribi l'altre metro de l'altra línia i pujar-hi en aquest. D'aquesta manera obliguem també als algorismes a que preguin les línies correctes quan es passa per aquests trams compartits. És a dir, si es vol anar de la parada L1 / L5 / L9N / L10N – La Sagrera a la L9N – Can Peixauet, que es prengui directament la L9N al tram compartit i no la L10N.

Hem dit que els resultats d'aquestes dues qüestions es complementen perquè en els de la primera (vegeu el Llistat 1) ja es dedueix que hem de tenir arestes diferents per les andanes compartides i en els de la segona (vegeu el Llistat 2) també es dedueix la necessitat del graf dirigit.

3.3.2 Introducció al cas particular de la parada L3 – Liceu

Un cas que cal destacar és el de la parada L3 – Liceu. Actualment és l'única estació de tota la xarxa en què, per canviar de sentit, és necessari sortir a la superfície i tornar a accedir a l'estació per una altra entrada [6, 2].

En el model adoptat, però, es considera un únic vèrtex per a cada andana, sense distingir el sentit de circulació. En conseqüència, les diferents entrades de Liceu s'associen al mateix vèrtex, malgrat que en la realitat cadascuna només permet accedir a un dels dos sentits.

Per mantenir la simplicitat del model i, alhora, reproduir correctament aquest cas particular, s'ha implementat una correcció comuna a tots els algorismes de camins mínims. Aquesta correcció es descriu a l'Annex C.1.

Una vegada resoltes aquestes decisions estructurals i analitzat aquest cas particular, només resta determinar quin pes s'assigna a cada aresta de tipus SW.

3.4 Assignació dels pesos SW

Partint d'aquí, el valor més simple i coherent que se'ns acut per SW és la mitjana de temps dels viatges entre cada andana. Tanmateix, és aquest valor suficientment representatiu de les tendències i normes implícites de la realitat que volem modelar?

Per tal de validar aquesta hipòtesi, s'ha desenvolupat el fitxer `edge_weight_validation.py`, que avalua la idoneïtat de la mitjana com a pes estàtic. Per fer-ho, s'utilitzen dues mètriques.

La primera és el coeficient de variació CV, definit com

$$CV = \frac{\sigma}{\mu},$$

on σ és la desviació estàndard i μ la mitjana dels temps observats. Aquesta mètrica mesura la dispersió relativa dels temps de recorregut (els valors baixos indiquen que les observacions es concentren al voltant de la mitjana).

La segona és el rang horari relatiu HR, definit en aquest treball com

$$HR = \frac{\max_h \mu_h - \min_h \mu_h}{\mu},$$

on μ_h és el temps mitjà observat a la franja horària h , $\max_h \mu_h$ i $\min_h \mu_h$ són, respectivament, el màxim i el mínim d'aquests valors entre totes les franges horàries considerades, i μ és la mitjana global dels temps observats. Aquesta mètrica quantifica fins a quin punt el temps de recorregut depèn de l'hora del dia. De nou, valors baixos indiquen una baixa dependència amb l'horari.

Els resultats mostren que el 99.69% de les arestes presenten una variabilitat acceptable ($CV < 15\%$) i que només el 7.81% es beneficiarien d'un model dependent de l'hora ($HR \geq 15\%$). En particular, l'única aresta amb una variabilitat inestable està inclosa dins del conjunt d'arestes a les que els beneficiaria pesos variables. Més detalls de l'estudi disponibles al Llistat 3 i a `edge_weight_validation_report.txt`.

Cal destacar que, en la majoria de casos, la variabilitat dels temps de recorregut es concentra en el temps que el metro permanence aturat a la primera andana amb les portes obertes, més que no pas en el temps de circulació entre estacions. Aquesta variabilitat, així com les diferències entre franges horàries, poden explicar-se per les fluctuacions de la demanda al llarg del dia i per incidències puntuals en el servei.

En conclusió, l'ús de pesos constants no només simplifica la construcció del graf i l'aplicació dels algoritmes de camí mínim, sinó que també constitueix una aproximació fidel al comportament de la xarxa. En particular, la mitjana dels temps observats proporciona un pes estàtic representatiu per al 92.19% de les arestes SW, motiu pel qual s'adopta com a criteri de ponderació en aquest treball.

3.5 Graf resultant

Com a resultat d'aquest procés s'ha construït el fitxer `weights.txt` que conté cada aresta del graf juntament amb el seu pes i el tipus d'aquest.

Finalment, hem obtingut un graf ponderat estrictament positiu, ja que és positiu i finit. Per tant, el pes mínim entre dos vèrtexs sempre existeix i es pot trobar un camí mínim entre ells.

A la Taula 3 s'han recollit les dades principals del graf. Té ordre 673 i mida de 1 419 arcs. Un factor prou important i identificatiu és que gairebé tres quartes parts dels vèrtexs són entrades i té un nombre molt baix d'arestes, el què el fa ser molt dispers.

A la Figura 2 es pot observar la disposició dels vèrtexs i les arestes del resultat final.

Taula 3: Principals característiques del graf construït

Vèrtexs	
Nombre total $ V $	673
↪ Entrades	502 (74.6%)
↪ Andanes	171 (25.4%)
↪ Vèrtexs duplicats	12 (1.8%)
↪ Entrades duplicades	0
↪ Andanes duplicades	12
Arcs	
Nombre total $ E $	1 419
↪ PW	1 019 (71.8%)
↪ TF	80 (5.6%)
↪ SW	320 (22.6%)
↪ Arcs duplicats	86 (6.1%)
↪ PW duplicats	32
↪ TF duplicats	28
↪ SW duplicats	26
Nombre d'arestes (no dirigides)	710
↪ PW	510
↪ TF	40
↪ SW	160
Capacitat	0.3%

Taula creada a partir de `graph_report.py`. Amb *Capacitat d'arcs* ens referim al nombre d'arcs comparat amb el nombre màxim que hi podria haver ($|V| \cdot (|V| - 1) = 452\,256$). Amb duplicació ens referim al tractament de les andanes compartides de les línies L9 i L10.

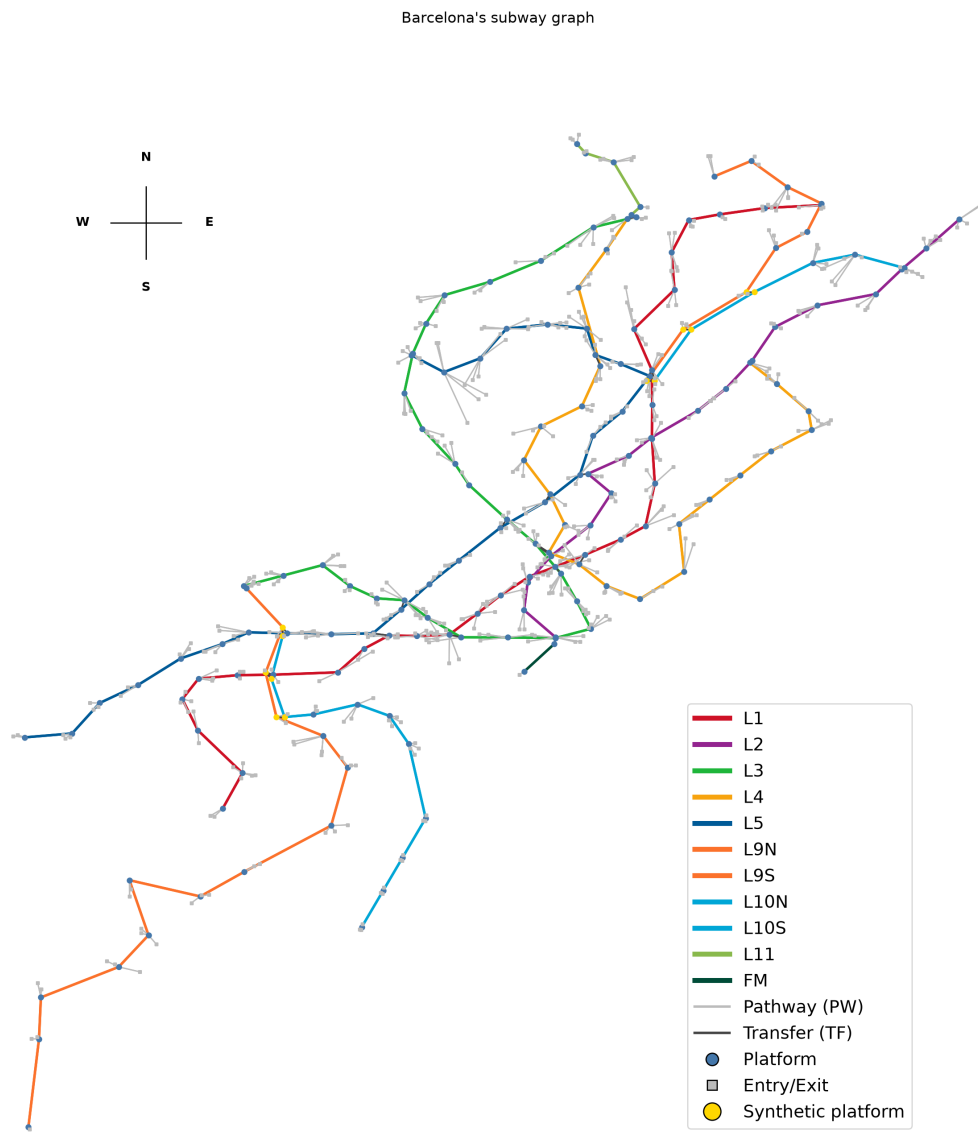


Figura 2: Graf construït. Figura generada amb `graph.py` a partir del fitxer `weights.txt`.

4 Cerca de camins mínims

4.1 Introducció

Una vegada construït el graf de la xarxa de metro, juntament amb la línia del Funicular de Montjuïc, l'objectiu és trobar camins mínims entre parelles d'entrades, simulant així les rutes que seguiria un usuari real. Com que els pesos de les arestes representen temps de recorregut, els camins mínims obtinguts no corresponen necessàriament a les rutes més curtes en distància, sinó a les més ràpides.

En aquest treball, una iteració correspon a una execució del bucle principal de l'algoritme de cerca, és a dir, a una extracció d'un vèrtex de la cua de prioritat i al processament dels seus successors (vegeu els algorismes 1 i 2). L'eficiència dels algorismes es mesurarà mitjançant el nombre d'iteracions necessàries per obtenir la solució òptima. Cal, però, distingir entre eficiència de la cerca i temps d'execució. Un algoritme que requereix menys iteracions pot considerar-se més eficient en el sentit que explora una part més petita del graf abans de trobar el camí mínim. Tanmateix, això no implica necessàriament que sigui més ràpid, ja que cada iteració pot comportar un cost computacional diferent segons les operacions que s'hi realitzen.

També convé diferenciar entre el nombre d'iteracions i el nombre de vèrtexs extrets de la cua de prioritat o explorats. En l'algoritme Dijkstra, aquestes dues quantitats coincideixen, ja que cada iteració extreu un vèrtex que no tornarà a inserir-se a la cua (Teorema 2.1). En canvi, en l'algoritme A* aquesta equivalència només està garantida quan la funció heurística és consistent (Observació 2.18). Si l'heurística és únicament admissible, un mateix vèrtex pot ser extret i reinserit diverses vegades, de manera que el nombre d'iteracions deixa de coincidir amb el nombre de vèrtexs diferents explorats (Observació 2.17).

Tot aquest mòdul es recolza en el directori `shortest_paths_algorithms` del repositori. Per més detalls tècnics sobre la implementació, es convida al lector a consultar-lo.

4.2 Dijkstra

La implementació de l'algoritme sobre el graf es troba al directori `shortest_paths_algorithms/dijkstra`, juntament amb alguns exemples sobre casos més simples que faciliten la comprensió (vegeu `dijkstra_example.py`).

A partir del pseudocodi de l'Algoritme 1 i de la teoria de la Secció 2.2.1, s'ha desenvolupat una implementació de l'algoritme Dijkstra sobre el graf construït a partir del fitxer `weights.txt`, a `dijkstra.py`.

Basant-nos en l'Observació 2.7, també s'ha implementat una versió de l'algoritme que s'atura tan bon punt s'obté el camí mínim fins a un node de destí donat. A aquesta variant l'anomenarem *Cut-Dijkstra*.

Com que l'objectiu del treball és modelitzar i optimitzar rutes entre un origen i un destí concrets, a partir d'aquest punt s'utilitzarà sempre el Cut-Dijkstra. Per simplicitat, quan el context no indueixi a confusió, ens hi referirem simplement com a Dijkstra.

Una vegada implementat l'algoritme, és interessant analitzar-ne el rendiment. En particular, com que el graf de la xarxa és relativament petit (vegeu la Taula 3), convé estudiar si el

Dijkstra ja resol les rutes amb un nombre prou reduït d'iteracions o si, pel contrari, la incorporació d'una funció heurística pot reduir significativament la cerca.

Per valorar aquesta opció, cal definir una mètrica que permeti quantificar l'eficiència de la cerca.

4.3 Rendiment dels algoritmes

La mètrica per quantificar l'eficiència o rendiment de la cerca hauria de ser independent de la longitud del trajecte. En efecte, no és equivalent necessitar 100 iteracions per obtenir un camí mínim format per 50 vèrtexs que necessitar-ne les mateixes per a un camí de només 5 vèrtexs.

Idealment, la major part dels vèrtexs explorats haurien de formar part del camí mínim retornat. Això motiva, doncs, a considerar la proporció

$$\frac{\text{Nre. vèrtexs al camí mínim}}{\text{Nre. iteracions}}.$$

Aquest quocient pren valors entre 0 i 1, i és més gran com més eficient és la cerca. En el cas ideal, tots els vèrtexs explorats pertanyerien al camí mínim i, per tant, el seu valor seria igual a 1. Tanmateix, aquest valor no sempre es pot assolir.

En efecte, l'algoritme que més s'hi aproxima és un A^* amb una heurística ideal, és a dir, una heurística que coneix exactament el cost mínim restant des de qualsevol vèrtex fins al destí. Tot i així, quan el camí mínim no és únic (Observació 2.2), la proporció pot ser inferior a 1. L'exemple de l'Annex C.2 il·lustra aquest cas.

En el nostre cas, però, aquesta situació és poc probable. La major part dels recorreguts transcorren per arestes de tipus SW, on els temps de recorregut acostumen a ser prou diversos, i per tant, els empats en la cua de prioritats seran poc freqüents. Per aquest motiu, és raonable prendre el valor 1 com a referència ideal a l'hora d'interpretar aquesta proporció.

Tot i conèixer aquest valor de referència, resulta útil disposar també d'un algoritme que s'hi approximi tant com sigui possible. Per aquest motiu, a continuació construirem una implementació de l' A^* amb una heurística ideal.

4.4 A^*_{cheat}

Definició 4.1 (Heurística ideal). L'heurística ideal h_{cheat} entre dos vèrtexs $n, t \in V$ es defineix com

$$h_{\text{cheat}}(n, t) = \sigma(n, t),$$

on $\sigma(n, t)$ denota el pes mínim entre n i t .

Observació 4.1. L'heurística h_{cheat} és la màxima heurística admissible possible. És a dir, si h és qualsevol altra heurística admissible, aleshores

$$h(n, t) \leq h_{\text{cheat}}(n, t), \quad \forall n, t \in V.$$

Observació 4.2. h_{cheat} és admissible i consistent trivialment.

Observació 4.3. El subíndex *cheat* fa referència al fet que aquesta heurística utilitza exactament el cost mínim restant fins al destí. En conseqüència, per construir-la cal conèixer prèviament la solució del problema que l'algoritme A^* intenta resoldre. Per aquest motiu, no es pot utilitzar com a heurística en una aplicació real i només s'emptra com a referència per comparar la qualitat d'altres heurístiques.

Observació 4.4. Una manera de construir aquesta heurística consisteix a executar prèviament l'algoritme Dijkstra (o el Cut-Dijkstra) des del node de destí. El Teorema 2.1 i l'Observació 2.10 garanteixen l'obtenció d'un camí mínim i el pes mínim entre el node d'origen i el de destí.

Notació 4.1. Utilitzarem A^*_{cheat} per referir-nos a l'algoritme A^* amb l'heurística h_{cheat} .

A partir del pseudocodi de l'Algoritme 2 i de la teoria de la Secció 2.2.2, s'ha desenvolupat una implementació de l'algoritme A^* sobre el graf construït a partir del fitxer `weights.txt`. Aquesta implementació es troba al fitxer `a_star.py`, mentre que la construcció de l'heurística h_{cheat} s'ha implementat a `h_cheat.py`.

Per facilitar la comprensió de l'algoritme, també s'ha desenvolupat un exemple senzill que utilitza una heurística diferent d' h_{cheat} . Tant aquest exemple com la resta de recursos relacionats es poden trobar al directori `shortest_paths_algorithms/a_star`.

4.5 Metodologia d'avaluació

Una vegada implementats els algoritmes Dijkstra i A^*_{cheat} per a qualsevol parell de nodes Source–Target, i definida la mètrica amb què els compararem (Secció 4.3), només resta establir la metodologia de comparació.

Com que la xarxa conté 502 entrades (Taula 3), comparar individualment totes les rutes possibles no resulta pràctic. En efecte, existeixen $502 \cdot 501 = 251\,502$ parelles ordenades diferents d'origen i destí (a l'apartat 3.3.1 ja hem vist la importància de la direcció del recorregut).

Atès que l'objectiu és estudiar el comportament global dels algoritmes i no casos particulars, s'ha decidit utilitzar com a indicador la mitjana de la mètrica definida anteriorment sobre totes aquestes rutes.

A més de calcular la mètrica, interessa conservar per a cada ruta informació addicional, com ara la Source, el Target, el camí mínim obtingut o el valor de la mètrica, entre d'altres, ja que aquestes dades seran útils en estudis i exemples posteriors. Emmagatzemar tota aquesta informació per a 251 502 rutes generaria, però, un fitxer molt pesat, de manera que convé plantejar una alternativa.

En lloc de considerar totes les entrades, agrupem les rutes per parelles d'andanes. Com que només n'hi ha 171, el nombre de rutes es redueix fins a $171 \cdot 170 = 29\,070$, obtenint així un document 8.65 vegades més curt.

Aquesta simplificació introdueix una petita distorsió en la proporció definida a la Secció 4.3 de les rutes més curtes, ja que s'empraran iteracions en explorar les entrades associades a les andanes d'origen, sense que aquestes es comptin en els vèrtexs del camí mínim. En conseqüència, el valor d'aquesta mètrica pot diferir lleugerament del que s'obtindria treballant

directament amb les entrades. Tanmateix, com que l'estudi considera un nombre molt elevat de rutes i l'indicador final és la mitjana sobre totes elles, aquest efecte és pràcticament negligible i no altera les conclusions generals.

Aquest estudi s'ha implementat al directori `shortest_paths_algorithms/reports`. Per a més detalls tècnics sobre la implementació, es convida el lector a consultar-lo.

Aplicant aquest procediment s'obté, finalment, una proporció mitjana aproximada de 0.06 pel Dijkstra i de 0.99 per l' A^*_{cheat} . El valor obtingut per aquest últim és coherent amb la discussió de la Secció 4.3, ja que només en casos molt excepcionals la proporció ideal 1 no s'assoleix.

En canvi, el resultat del Dijkstra mostra que, tot i garantir sempre un camí mínim, explora molts més vèrtexs dels estrictament necessaris abans d'arribar al destí.

Això motiva la cerca d'una heurística que orienti l'algoritme cap al Target i redueixi el nombre de vèrtexs explorats, però sense disposar de la informació exacta que utilitza h_{cheat} .

4.6 A^*_{geo}

A l'hora de construir una heurística que no sobreestimi el cost mínim restant fins al Target, és natural considerar la distància en línia recta entre cada node i el destí. Tanmateix, com que els pesos del nostre graf representen temps de recorregut i no distàncies, cal transformar aquesta magnitud en una estimació temporal.

4.6.1 Definició de l'heurística h_{geo}

Definició 4.2 (Distància geogràfica). La distància geogràfica en línia recta entre dos nodes a i b es defineix com

$$d_{\text{geo}}(a, b) = R\sqrt{(\Delta\lambda \cos \bar{\phi})^2 + (\Delta\phi)^2},$$

on R és el radi de la Terra, $\Delta\lambda$ i $\Delta\phi$ són les diferències de longitud i latitud (en radians), respectivament, i $\bar{\phi}$ és la latitud mitjana entre els dos punts. Aquesta expressió correspon a l'aproximació equirectangular, basada en una projecció cilíndrica equidistant [5, p. 90].

Observació 4.5. Atès que la xarxa estudiada es troba íntegrament dins l'àrea metropolitana de Barcelona, la curvatura de la Terra és negligible a l'escala considerada. Per tant, es pot assumir que la superfície és plana sense introduir un error apreciable en els càlculs de distància.

Observació 4.6. Entre les diferents aproximacions planes possibles, s'ha optat per la projecció equirectangular en lloc d'aplicar directament el teorema de Pitàgores sobre les coordenades de latitud i longitud. Aquesta projecció corregeix la diferent escala de la longitud segons la latitud mitjançant el factor $\cos(\bar{\phi})$, obtenint una estimació més precisa de la distància sense incrementar de manera significativa el cost computacional.

Definició 4.3 (Velocitat màxima al graf). Es defineix la velocitat màxima implícita al graf com

$$v_{\text{max}} = \max_{(u,v) \in E} \frac{d_{\text{geo}}(u, v)}{\omega(u, v)},$$

on $\omega(u, v)$ representa el temps de recorregut de l'arc (u, v) .

Observació 4.7. En particular, al nostre graf la velocitat màxima, aproximadament, és de 11.2 m/s (40.4 km/h) i es dona a l'arc de la parada L9S – Mercabarna a la L9S – Parc Logístic.

Definició 4.4 (Heurística geogràfica). L'heurística geogràfica h_{geo} entre dos vèrtexs $n, t \in V$ es defineix com

$$h_{\text{geo}}(n, t) = \left\lfloor \frac{d_{\text{geo}}(n, t)}{v_{\text{max}}} \right\rfloor.$$

Observació 4.8. Cal denotar que l'operació **floor** $\lfloor \cdot \rfloor$ fa que h_{geo} retorni nombres enters, en aquest cas segons.

Proposició 4.1. L'heurística h_{geo} és admissible.

Demostració. Considerem un camí qualsevol

$$\alpha = v_0 \rightarrow v_1 \rightarrow \cdots \rightarrow v_n$$

entre el node d'origen $\xi := v_0$ i el node de destí $\gamma := v_n$.

Per definició,

$$h_{\text{geo}}(\xi, \gamma) = \left\lfloor \frac{d_{\text{geo}}(\xi, \gamma)}{v_{\text{max}}} \right\rfloor.$$

Com que $\lfloor x \rfloor \leq x$ per a tot $x \in \mathbb{R}$,

$$h_{\text{geo}}(\xi, \gamma) \leq \frac{d_{\text{geo}}(\xi, \gamma)}{v_{\text{max}}}.$$

Aplicant la desigualtat triangular,

$$\frac{d_{\text{geo}}(\xi, \gamma)}{v_{\text{max}}} \leq \frac{d_{\text{geo}}(\xi, v_1) + d_{\text{geo}}(v_1, v_2) + \cdots + d_{\text{geo}}(v_{n-1}, \gamma)}{v_{\text{max}}}.$$

Per la definició de v_{max} ,

$$\frac{d_{\text{geo}}(u, v)}{\omega(u, v)} \leq v_{\text{max}} \quad \forall (u, v) \in E,$$

i, equivalentment,

$$\frac{d_{\text{geo}}(u, v)}{v_{\text{max}}} \leq \omega(u, v).$$

Aplicant aquesta desigualtat a cadascuna de les arestes del camí α , obtenim

$$\frac{d_{\text{geo}}(\xi, v_1) + d_{\text{geo}}(v_1, v_2) + \cdots + d_{\text{geo}}(v_{n-1}, \gamma)}{v_{\text{max}}} \leq \sum_{i=1}^n \omega(v_{i-1}, v_i).$$

Per tant,

$$h_{\text{geo}}(\xi, \gamma) \leq \omega(\alpha).$$

Com que aquesta desigualtat és certa per a qualsevol camí α entre ξ i γ , també és certa prenent el mínim sobre tots ells:

$$h_{\text{geo}}(\xi, \gamma) \leq \min_{\alpha} \omega(\alpha) = \sigma(\xi, \gamma).$$

En conseqüència, h_{geo} és admissible. □

Notació 4.2. Utilitzarem A^*_{geo} per referir-nos a l'algoritme A^* amb l'heurística h_{geo} .

Observació 4.9. Observem que, per a tot $n, t \in V$,

$$h_{\text{cheat}}(n, t) \geq h_{\text{geo}}(n, t) \geq h_0(n, t). \quad (1)$$

Això fa esperar que el rendiment de l' A^*_{cheat} , l' A^*_{geo} i el Dijkstra segueixi el mateix ordre. Aquesta hipòtesi es validarà experimentalment a les Seccions 4.6.2, 4.7.4, 4.8.2 i 4.9.2.

La implementació de l'heurística h_{geo} es troba al fitxer `h_geo.py`.

4.6.2 Avaluació experimental

Finalment, aplicant el procediment descrit a l'apartat 4.5, obtenim una mitjana per l' A^*_{geo} de 0.12 (recordem que havíem obtingut un 0.06 pel Dijkstra i un 0.99 per l' A^*_{cheat}). Aquests valors concorden amb la cadena de desigualtats (1) i validen la nostra hipòtesi plantejada d'obtenir millors resultats amb aquest nou algoritme (Observació 4.9). De fet, no només s'ha millorat la mètrica anterior, sinó que s'ha de reconèixer que l'heurística h_{geo} ha duplicat el rendiment que teníem fins ara.

Tot i així, aquest valor encara segueix sent prou baix i encara hi ha un marge de millora molt gran. El que ens porta a pensar quin podria ser el següent pas per reduir el nombre d'iteracions.

Fins aquest moment, hem aplicat i construït els dos algoritmes que coneixem per resoldre el problema del camí mínim entre un origen i un destí, amb el segon tenint una heurística fàcil de deduir i aplicable a qualsevol graf. És ara, doncs, quan hem de parar-nos a pensar més enllà del que es sol fer en aquests tipus de problemes i centrar-nos en el nostre en particular per veure què podem fer de diferent.

4.7 Topologia particular del graf

4.7.1 Motivació

Remuntant-nos a la Secció 3.5 i, en particular, a la Taula 3, es poden identificar diverses característiques estructurals del graf que fins ara no s'han explotat en el disseny de les heurístiques.

En concret, destaquen els fets següents.

1. El 74.6% dels vèrtexs corresponen a entrades. A més, cada entrada només està connectada amb una única andana, excepte en el cas de les andanes duplicades de les línies L9 i L10, on una entrada pot connectar amb dues andanes (vist a l'Annex B.1.3).

2. Tot i que el graf només conté un 0.3% dels arcs possibles, un 71.8% dels arcs existents són de tipus PW.
3. A la Figura 2 es pot observar com la xarxa presenta una estructura molt característica: un nucli central connectat del qual emergeixen vuit branques.

Aquestes observacions tenen dues conseqüències immediates.

1. Les entrades només poden formar part d'un camí mínim quan corresponen al node Source o al node Target. Durant la resta del recorregut, l'usuari mai no les utilitza per desplaçar-se per la xarxa.
2. Els algoritmes estudiats fins ara (Dijkstra i A_{geo}^*) tracten tots els vèrtexs de la mateixa manera i, per tant, no aprofiten aquesta estructura tan marcada del graf.

Això explica els resultats obtinguts fins ara. D'una banda, una gran part de les iteracions es dedica a explorar entrades que, en realitat, no poden formar part del camí mínim (fet fàcilment observable en qualsevol execució del Dijkstra o de l' A_{geo}^*). Per l'altra, cap dels dos algoritmes és conscient que la xarxa està formada per un nucli central i diverses branques, de manera que continuen explorant moltes direccions que, estructuralment, difícilment poden conduir al destí.

Aquesta observació suggereix dividir el graf en regions que capturin aquesta estructura. A continuació formalitzem aquesta partició.

4.7.2 Definició de les regions

Definició 4.5 (Regions del graf). Les regions del graf són la partició del conjunt de vèrtexs

$$V = \text{Center} \sqcup \text{Branch}_{L1} \sqcup \text{Branch}_{L2} \sqcup \text{Branch}_{L5} \sqcup \text{Branch}_{L9S} \\ \sqcup \text{Branch}_{L9N} \sqcup \text{Branch}_{L10S} \sqcup \text{Branch}_{L11} \sqcup \text{Branch}_{FM}.$$

Per a cada branca, es consideren totes les andanes compreses entre les parades inicial i final indicades a la Taula 4, juntament amb totes les entrades adjacents a aquestes andanes.

La regió **Center** està formada per tots els vèrtexs que no pertanyen a cap de les branques.

Taula 4: Definició de les branques del graf

Branca	Parada inicial	Parada final
Branch_{L1}	L1 – Hospital de Bellvitge	L1 – Florida
Branch_{L2}	L2 – Badalona Pompeu Fabra	L2 – Pep Ventura
Branch_{L5}	L5 – Cornellà Centre	L5 – Ernest Lluch
Branch_{L9S}	L9S – Aeroport T1	L9S – Europa Fira
Branch_{L9N}	L9N – Can Zam	L9N – Església Major
Branch_{L10S}	L10S – ZAL Riu Vell	L10S – Provençana
Branch_{L11}	L11 – Can Cuiàs	L11 – Casa de l'Aigua
Branch_{FM}	FM – Parc Montjuïc	FM – Parc Montjuïc

Definició 4.6 (Pont). Per a cada branca definim el seu pont com l'única andana de la regió Center que està connectada mitjançant una aresta amb alguna andana d'aquella branca.

En el nostre cas, aquests ponts són els de la Taula 5 .

Taula 5: Pont associat a cada branca

Branca	Pont
Branch_{L1}	L1 – Torrassa
Branch_{L2}	L2 – Gorg
Branch_{L5}	L5 – Collblanc
Branch_{L9S}	L9S – Can Tries Gornal
Branch_{L9N}	L9N – Fondo
Branch_{L10S}	L10S – Can Tries Gornal
Branch_{L11}	L11 – Trinitat Nova
Branch_{FM}	FM – Paralel

Aquesta partició reflecteix a la Figura 3 la forma natural de la xarxa: un nucli central que concentra la major part dels vèrtexs i vuit branques que només s'hi connecten a través d'un únic pont.

4.7.3 Casos segons les regions

La Definició 4.5 permet classificar qualsevol parell ordenat de vèrtexs segons les seves regions.

Definició 4.7 (Casos segons les regions). Siguin $C = \text{Center}$ i $\mathcal{B}$ el conjunt de totes les branques de la Definició 4.5. Definim els subconjunts següents de $V \times V$:

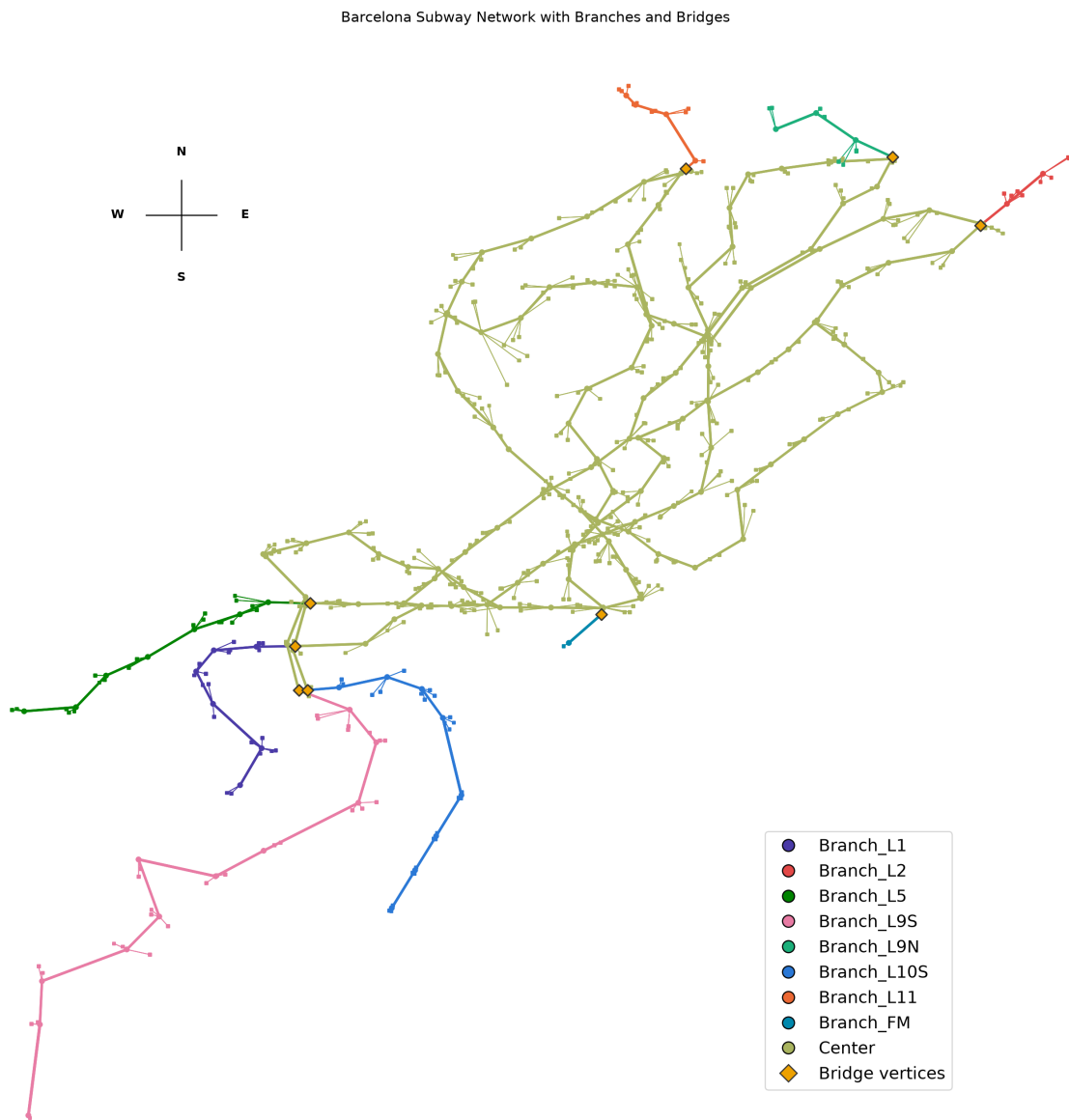
$$\begin{aligned}
CC &= \{(u, v) \in V \times V \mid u, v \in C\}, \\
SB &= \{(u, v) \in V \times V \mid \exists B \in \mathcal{B} : u, v \in B\}, \\
CB &= \{(u, v) \in V \times V \mid u \in C, \exists B \in \mathcal{B} : v \in B\}, \\
BC &= \{(u, v) \in V \times V \mid \exists B \in \mathcal{B} : u \in B, v \in C\}, \\
DB &= \{(u, v) \in V \times V \mid \exists B_1, B_2 \in \mathcal{B} : B_1 \neq B_2, u \in B_1, v \in B_2\}.
\end{aligned}$$

Observació 4.10. Els noms venen de *center to center*, *same branch*, *center to branch*, *branch to center* i *different branches*, seguint el mateix ordre.

Observació 4.11. Els cinc casos anteriors formen una partició del conjunt $V \times V$. En particular, qualsevol parell ordenat de vèrtexs pertany a un únic cas.

4.7.4 Validació experimental i anàlisi

La introducció de les particions de les definicions 4.5 i 4.7 només són útils si realment expliquen una part significativa del comportament dels algorismes. Per validar-ho, repetim



* Synthetic platform and entry/exit positions are artificially offset, not to real scale, for visual clarity.

Figura 3: Regions i ponts del graf. Figura generada amb `regions_graph.py` a partir de la definició implementada a `barcelona_division.py`.

l'estudi de rendiment de les Seccions 4.5 i 4.6.2, però distingint ara els resultats segons els cinc casos definits anteriorment pel parell ordenat de Source i Target (ξ, γ) . Aquesta divisió es pot observar a la Taula 6.

Taula 6: Rendiment mitjà de cerca del Dijkstra, l' A^*_{geo} i l' A^*_{cheat} segons el cas $(\xi, \gamma) \in V \times V$. Taula creada a partir de `algorithms_comparison.py`.

Cas	n	%	Cut-Dijkstra	A^*_{geo}	A^*_{cheat}
CC	16 512	56.80	0.06	0.14	1
CB	5 418	18.64	0.04	0.07	0.99
BC	5 418	18.64	0.07	0.12	0.99
SB	258	0.89	0.25	0.52	1
DB	1 464	5.04	0.07	0.12	0.99
Global	29 070	100	0.06	0.12	0.99

El fet que el rendiment mitjà de l' A^*_{geo} augmenti, aproximadament, 7.4 vegades en passar del cas CB al cas SB evidencia que la topologia del graf és un factor determinant i que, per tant, no es pot ignorar en el disseny d'una heurística.

A més, la cadena de desigualtats (1) continua reflectint-se en els resultats experimentals. No només s'observa globalment que l' A^*_{cheat} supera l' A^*_{geo} i aquest supera el Dijkstra, com ja havíem mencionat prèviament a la Secció 4.6.2, sinó que aquesta mateixa jerarquia es manté dins de cadascun dels cinc casos.

El cas SB és, amb diferència, aquell on s'obtenen les millors mètriques. Aquest resultat és esperable, ja que quan l'origen i el destí pertanyen a una mateixa branca hi ha moltes menys alternatives de recorregut i, en conseqüència, l'espai de cerca és molt més reduït.

Observem també que hi ha una clara diferència en la simetria dels casos BC i CB. Aquest fenomen és senzill d'imaginar i entendre si pensem Barcelona com un rectangle (abaix es troba el mar i adalt la montanya) amb una línia vermella vertical. Si la ruta comença a una branca això significa que la Source es troba a una punta del graf. És a dir, la línia vermella està a prop d'una de les vores. El Dijkstra acabarà explorant part de la petita zona entre la línia vermella i la vora propera pel seu comportament greedy. A mesura que avancin les iteracions, la línia vermella anirà avançant en el rectangle amb direcció a l'altra vora. En el moment que la línia s'apropi al Target, l'algoritme haurà explorat els nodes fins arribar allà. Si ens ho imaginem ara en direcció contrària, tenim que la línia comença aproximadament centrada en el rectangle. Al no tenir cap noció de cap a on pot trobar-se la Source, el Dijkstra explorarà nodes tant com per una banda com per l'altra. En el moment d'arribar al destí, l'algoritme haurà passat per gairebé tota la figura.

És a dir, com el Dijkstra explora tant per davant com per darrere de la Source, es gastaran menys iteracions quan aquesta es trobi en una punta, que no pas si es troba al mig.

Això es veu clarament representat a l'exemple de la Figura 9 i de la Figura 10.

En el cas de l' A^*_{geo} , sí que és cert que no s'arriben a explorar tants nodes cap a endarere de la Source quan es troba al centre, però per què tot i així se segueixen explorant alguns de darrere si aquest algoritme en particular té una mínima noció de la posició del destí? És

a dir, per què s'extreuen nodes de la pila que es troben en direcció contrària abans que uns més propers a la Source?

Bé, això es pot explicar per la (baixa) aproximació del cost real que té l'heurística. Per il·lustrar-ho, tornarem al mateix exemple del cas CB.

Si busquem les iteracions concretes al seu informe, veiem que s'explora abans la parada L2 – Verneda (iteració 357) que la L1 – Bellvitge (iteració 362). Ens podem preguntar com és possible si Bellvitge es troba a una parada del destí i Verneda encara és més lluny que l'origen.

Els valors utilitzats per l'algoritme són els següents:

	g	h	$f = g + h$
L2 – Verneda	590	1144	1734
L1 – Bellvitge	1668	72	1740

Tot i que Bellvitge és molt més propera al destí, el cost acumulat per arribar-hi (g) és molt superior. La diferència entre els valors heurístics (h) no és suficient per compensar aquesta situació i, per tant, Verneda obté un valor f inferior ($1734 < 1740$) i és extreta abans de la cua de prioritat.

És a dir, perquè Bellvitge hagués estat explorada abans, l'heurística hauria d'haver assignat a Verneda una estimació més gran, més propera al cost real que encara resta fins al destí, en lloc d'una cota tan optimista.

Amb això podem veure com, fins i tot tenint noció de la distància (en línia recta), aquesta no és una cota suficientment bona i pot provocar que a ulls de l'algoritme es distorsionin les magnituds de les distàncies reals.

Llavors, pel mateix argument de que començant al mig tenim més nodes disponibles al voltant, té sentit que l' A_{geo}^* operi pitjor en el cas CB que no pas BC.

Apart d'ajudar-nos a entendre la diferència entre aquests dos casos, a l'exemple particular de CB, també hem pogut veure com s'exploren bastants nodes al centre els quals no acaben formant part del camí mínim més tard. Això, de fet, succeeix en tots els casos en els que al camí mínim ha de pertànyer un vèrtex $v \in V$ tal que el parell (v, γ) pertany als casos CC, CB, BC i DB. Aquests, de fet, representen el 99.11% del total.

També, ambdós algoritmes exploren nodes a branques en les què no es troba el destí. En particular, de nou a l'exemple de la Figura 10, veiem com l' A_{geo}^* visita les parades L5 – Can Vidalet, L9S – Fira i L10S – Ciutat de la Justícia perquè es troben una distància curta en línia recta del destí. La realitat és que aquest algoritme no contempla les nocions de topologia definides en aquest apartat, i és per això que no sap identificar que per saltar el forat que hi ha entre les branques primer ha de fer marxa enrere per tornar al centre. Per millorar això, doncs, hauríem de construir una heurística que sí tingués consciència d'aquesta forma de la xarxa.

A més, als dos exemples que hem vist de CB i BC podem observar com es perden moltes iteracions explorant entrades que no són ni la inicial ni la final. Això, de fet, passa a tots els casos i es pot comprovar mirant els informes de les iteracions dels algoritmes.

A banda de comprovar que, efectivament, la topologia particular del graf és rellevant i

d'entendre per què obtenim diferents mètriques a cada cas, podem dir que se'ns han plantejat quatre problemes a resoldre:

- **P1** – S'exploren nodes al voltant del Target en direcció contrària d'on es troba la Source
- **P2** – S'exploren molts nodes al centre que finalment no formen part del camí mínim
- **P3** – Quan el Target es troba en una branca, s'exploren nodes a branques diferents, amb la Source no pertanyent allà
- **P4** – La majoria d'iteracions es perden en entrades que no són ni la Source ni el Target

Veurem si amb les millores implementades a partir d'ara es resolen i com.

4.8 A_{bcn}^*

A partir de la topologia particular del graf estudiada a la secció anterior, és natural plantejar una funció heurística que exploti aquesta estructura. En particular, la definició de les regions, els seus ponts i els diferents casos segons la posició del parell (ξ, γ) permeten incorporar informació que fins ara no utilitzaven ni el Dijkstra ni l' A_{geo}^* .

Per exemple, si volem anar de Barcelona a Sevilla, una aproximació basada en la distància en línia recta pot ser raonable. En canvi, si volem anar de Barcelona a Roma, és evident que el trajecte haurà de passar per determinades zones intermèdies (com Toulouse i Milà), i una bona estimació hauria de tenir en compte aquests punts de pas¹. De manera anàloga, en la nostra xarxa els ponts entre regions actuen com aquests punts obligatoris.

A més, aquesta estructura també suggereix la possibilitat de tractar de manera diferent els vèrtexs d'entrada, atès que, com s'ha vist, només formen part d'un camí mínim quan corresponen a la Source o al Target.

4.8.1 Definició de l'heurística h_{bcn}

Definició 4.8 (Heurística barcelonina). L'heurística barcelonina h_{bcn} entre dos vèrtexs $n, t \in V$ es defineix com

$$h_{\text{bcn}}(n, t) = \begin{cases} 0, & \text{si } n = t, \\ \infty, & \text{si } P_f(n) \text{ o } P_t(t) \text{ són buits,} \\ c_{\text{ent}}(n) + \min_{p \in P_f(n), q \in P_t(t)} h_{\text{reg}}(p, q) + c_{\text{ent}}(t), & \text{altrament,} \end{cases}$$

on

¹Exemple presentat sovint a les classes del Lluís Alseda

- $P_f(n)$ és el conjunt d'andanes accessibles des de n (*platforms from*). És a dir, si n és una andana, $P_f(n) = \{n\}$, i si n és una entrada, $P_f(n)$ és el conjunt d'andanes a les quals aquesta dona accés.
- $P_t(t)$ és el conjunt d'andanes que connecten amb t (*platforms to*). És a dir, si t és una andana, $P_t(t) = \{t\}$, i si t és una entrada, $P_t(t)$ és el conjunt d'andanes des de les quals es pot accedir a aquesta.
- $c_{\text{ent}}(\cdot)$ és el cost d'entrada, que val 60s si el node és una entrada i 0 si és una andana (comentat a la Secció 3.2 i vist a l'Annex B.1.3).
- $h_{\text{reg}}(p, q)$ és una heurística entre andanes definida com

$$h_{\text{reg}}(p, q) = \begin{cases} h_{\text{geo}}(p, q), & \text{si } (p, q) \in \text{CC}, \\ d_f(q) - d_f(p), & \text{si } (p, q) \in \text{SB i } d_f(p) < d_f(q), \\ d_t(p) - d_t(q), & \text{si } (p, q) \in \text{SB altrament}, \\ h_{\text{geo}}(p, b_q) + d_f(q), & \text{si } (p, q) \in \text{CB}, \\ d_t(p) + h_{\text{geo}}(b_p, q), & \text{si } (p, q) \in \text{BC}, \\ d_t(p) + h_{\text{geo}}(b_p, b_q) + d_f(q), & \text{si } (p, q) \in \text{DB}. \end{cases}$$

on b_p i b_q són els ponts de les branques a les quals pertanyen p i q , respectivament (*bridge*). A més, $d_f(v)$ denota el cost exacte des del pont de la branca fins a v (*distance from bridge*), mentre que $d_t(v)$ denota el cost exacte des de v fins al pont de la seva branca (*distance to bridge*).

Observació 4.12. h_{geo} és la de la Definició 4.4, els cinc casos segons les regions són els de la Definició 4.7 i els ponts els de la Definició 4.6.

Observació 4.13. d_f i d_t es calculen a partir dels pesos ω obtinguts a la Secció 3.5, i en particular són nombres enters (segons).

Observació 4.14. Excepte quan $P_f(n)$ o $P_t(t)$ són buits, h_{bcn} retorna nombres enters, en particular segons.

Observació 4.15. A diferència de l'heurística h_{cheat} , el càlcul dels valors d_f i d_t en l'heurística h_{reg} (d'andana a andana) no proporciona informació sobre els camins mínims del graf. En efecte, cada branca és un subgraf sense bifurcacions: entre qualsevol parell de les seves andanes només existeix un únic camí possible. Per tant, calcular d_f i d_t consisteix únicament a acumular els pesos de les arestes d'aquest camí únic, sense haver de resoldre cap problema de camins mínims.

En canvi, al nucli de la xarxa existeixen múltiples rutes alternatives. És precisament en aquesta part on determinar el pes mínim entre dos vèrtexs requereix executar un algoritme de camins mínims, i és això el que fa que h_{cheat} utilitzi informació de la qual un algoritme no hauria de disposar.

Observació 4.16. La definició de l'heurística té en compte que el graf és dirigit. Per aquest motiu, els conjunts P_f i P_t no coincideixen necessàriament, ja que una entrada pot donar accés a una determinada andana sense que des d'aquesta es pugui arribar novament a la mateixa entrada, o a l'inrevés.

De manera anàloga, les funcions d_f i d_t es defineixen separatament perquè el cost des del pont d'una branca fins a un vèrtex no té per què coincidir amb el cost en sentit contrari. Aquesta asimetria és també la que fa necessari distingir dos subcasos dins de SB en la definició d' h_{reg} . En conseqüència, l'heurística h_{bcn} no depèn de propietats específiques de la xarxa de Barcelona, sinó únicament de disposar d'un graf dirigit amb una partició en branques i una definició dels seus ponts.

Per demostrar l'admissibilitat d' h_{bcn} necessitarem primer un altre resultat.

Proposició 4.2. L'heurística h_{reg} és admissible.

Demostració. Considerem un parell ordenat qualsevol d'andanes $(p, q) \in V \times V$. Volem demostrar que

$$h_{\text{reg}}(p, q) \leq \sigma(p, q).$$

Considerem els diferents casos de la definició d' h_{reg} .

- Si $(p, q) \in \text{CC}$, aleshores

$$h_{\text{reg}}(p, q) = h_{\text{geo}}(p, q),$$

que és admissible per la Proposició 4.1.

- Si $(p, q) \in \text{SB}$, tant

$$d_f(q) - d_f(p)$$

com

$$d_t(p) - d_t(q)$$

representen exactament el cost del recorregut entre p i q dins de la mateixa branca. Per tant,

$$h_{\text{reg}}(p, q) = \sigma(p, q),$$

i l'heurística és admissible.

- Si $(p, q) \in \text{CB}$, es té

$$h_{\text{reg}}(p, q) = h_{\text{geo}}(p, b_q) + d_f(q).$$

Per la Proposició 4.1,

$$h_{\text{geo}}(p, b_q) \leq \sigma(p, b_q),$$

mentre que $d_f(q)$ és el cost exacte entre b_q i q . En conseqüència,

$$h_{\text{reg}}(p, q) \leq \sigma(p, b_q) + d_f(q) = \sigma(p, q).$$

- Si $(p, q) \in \text{BC}$, de manera anàloga,

$$h_{\text{reg}}(p, q) = d_t(p) + h_{\text{geo}}(b_p, q),$$

on $d_t(p)$ és el cost exacte entre p i b_p , i

$$h_{\text{geo}}(b_p, q) \leq \sigma(b_p, q).$$

Per tant,

$$h_{\text{reg}}(p, q) \leq d_t(p) + \sigma(b_p, q) = \sigma(p, q).$$

- Finalment, si $(p, q) \in \text{DB}$,

$$h_{\text{reg}}(p, q) = d_t(p) + h_{\text{geo}}(b_p, b_q) + d_f(q).$$

Els termes $d_t(p)$ i $d_f(q)$ són costos exactes, mentre que

$$h_{\text{geo}}(b_p, b_q) \leq \sigma(b_p, b_q).$$

Així,

$$h_{\text{reg}}(p, q) \leq d_t(p) + \sigma(b_p, b_q) + d_f(q) = \sigma(p, q).$$

En tots els casos,

$$h_{\text{reg}}(p, q) \leq \sigma(p, q),$$

i, per tant, h_{reg} és admissible. □

Proposició 4.3. L'heurística h_{bcn} és admissible.

Demostració. Considerem un parell qualsevol de nodes n i t .

Si $n = t$, significa que el node d'origen coincideix amb el node de destí. En aquest cas, per definició,

$$h_{\text{bcn}}(n, t) = 0.$$

D'altra banda, el camí mínim d'un node a ell mateix també té cost nul, és a dir,

$$\sigma(n, t) = 0.$$

En conseqüència,

$$h_{\text{bcn}}(n, t) = \sigma(n, t),$$

i l'heurística és admissible en aquest cas.

Suposem ara que $n \neq t$.

Si $P_f(n) = \emptyset$ o $P_t(t) = \emptyset$, no existeix cap camí entre n i t , de manera que tant l'heurística com el cost mínim valen ∞ .

Suposem, ara que $P_f(n)$ i $P_t(t)$ són no buits. Per definició,

$$h_{\text{bcn}}(n, t) = c_{\text{ent}}(n) + \min_{p \in P_f(n), q \in P_t(t)} h_{\text{reg}}(p, q) + c_{\text{ent}}(t),$$

on els termes $c_{\text{ent}}(n)$ i $c_{\text{ent}}(t)$ representen exactament el cost necessari per entrar i sortir de la xarxa de metro, respectivament.

Per la proposició anterior, h_{reg} és admissible. Així, per a qualsevol parell

$$(p, q) \in P_f(n) \times P_t(t),$$

es compleix

$$h_{\text{reg}}(p, q) \leq \sigma(p, q).$$

Com que el mínim d'un conjunt de cotes inferiors continua sent una cota inferior,

$$\min_{p \in P_f(n), q \in P_t(t)} h_{\text{reg}}(p, q)$$

no sobreestima el cost mínim entre les andanes accessibles des de n i les andanes connectades amb t .

Finalment, en sumar els costos exactes d'entrada i sortida, s'obté que

$$h_{\text{bcn}}(n, t) \leq \sigma(n, t),$$

i, en conseqüència, h_{bcn} és admissible. □

Notació 4.3. Utilitzarem A^*_{bcn} per referir-nos a l'algoritme A^* amb l'heurística h_{bcn} .

Observació 4.17. Observem que $\forall n, t \in V$

$$h_{\text{reg}}(n, t) \geq h_{\text{bcn}}(n, t) \geq h_{\text{geo}}(n, t) \geq h_0(n, t). \quad (2)$$

Això fa esperar que el rendiment de l' A^*_{cheat} , l' A^*_{bcn} , l' A^*_{geo} i el Dijkstra segueixi el mateix ordre.

Aquesta hipòtesi es validarà experimentalment a les Seccions 4.8.2 i 4.9.2.

La implementació de l'heurística h_{bcn} es troba al fitxer `h_bcn.py`.

4.8.2 Avaluació experimental i anàlisi

Per comprovar experimentalment la hipòtesi plantejada a l'observació anterior, repetim la mateixa metodologia d'avaluació emprada a les Seccions 4.5, 4.6.2 i 4.7.4. En particular, comparem la proporció mitjana definida a la Secció 4.3, tant globalment com distingint els cinc casos de la Definició 4.7.

Amb aquest procediment obtenim el gràfic de la Figura 4.

Primer de tot, cal destacar que, tal com ja havia succeït amb l' A^*_{geo} a les Seccions 4.6.2 i 4.7.4, l' A^*_{bcn} no només obté el millor rendiment global, sinó també en cadascun dels cinc casos. Aquest resultat confirma la predicció de la desigualtat (2). En particular, el seu rendiment global és 3.17 vegades superior al del Dijkstra i 1.58 vegades superior al de l' A^*_{geo} . Això reforça la idea que adaptar una heurística a les característiques específiques del nostre graf permet obtenir millores significatives respecte a una heurística de propòsit general.

Algorithm performance by graph region

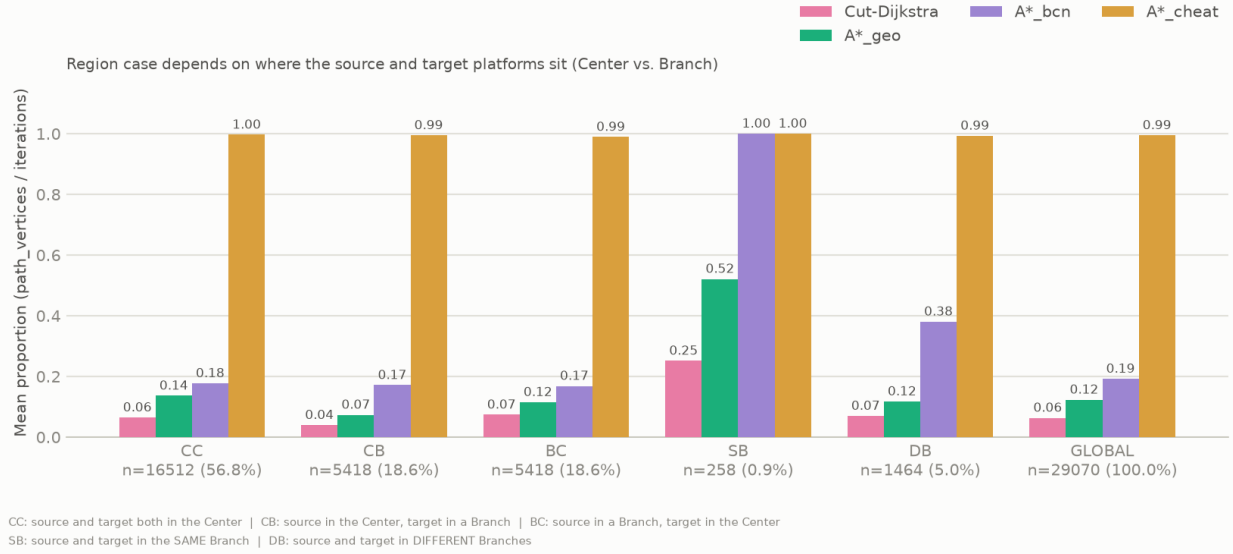


Figura 4: Rendiment mitjà de cerca del Dijkstra, l'A*_geo, l'A*_bcn i l'A*_cheat segons el cas $(\xi, \gamma) \in V \times V$. Figura generada amb `algorithms_comparison.py`.

Analitzem ara les diferències que introdueix l'A*_bcn respecte de l'A*_geo en cadascun dels casos i expliquem per què es produeixen.

El cas SB és l'únic en què s'assoleix el rendiment perfecte. Això es deu al fet que per nodes a la mateixa branca que el Target, h_{bcn} dona exactament el pes mínim σ entre ells. Ho veurem més endavant, però això fa que el problema P4 s'hagi solucionat en aquest cas.

La millora a CC, per la definició de h_{bcn} , es deu exclusivament al cost d'entrada c_{ent} . Encara que h_{bcn} intenti solucionar el problema P4 plantejat a l'Apartat 4.7.4 amb aquesta penalització, es segueixen explorant entrades que no ens interessin (a la Figura 11 es pot apreciar això). De fet, evitar-les per complet intentant augmentar el cost d'entrada és impossible perquè llavors l'heurística deixaria de ser admissible (si es pensa el cas de la ruta entre dues entrades d'una mateixa andana es veu clar). En conseqüència, P4 només s'ha atenuat mínimament. De fet, al mateix exemple de la figura plantejat, encara que la ruta sigui curta es pot apreciar com els problemes P1 i P2 segueixen presents.

On podem observar una millora més gran respecte A*_geo és al cas DB, multiplicant per 3.17 vegades el rendiment que teníem fins ara. Vegem un parell d'exemples per entendre-ho.

A la Figura 12 podem veure com l'A*_bcn només utilitza les iteracions exclusivament necessàries per arribar al Target. Cal recalcar, però, que encara que aquest sigui tècnicament un exemple del cas DB, a la pràctica s'assembla molt a un de SB. És per això que no tenim cap entrada innecessària explorada per l'A*_bcn, tal i com hem mencionat més amunt. Tanmateix, aquí no podem dir res del problemes P1 i P2, però sí que podem observar que el P3 no mostra senyals d'existir.

A la Figura 13 podem veure el cas contrari al previ a DB. En el que primer ens fixem és que la major part d'iteracions s'han perdut al centre (aquest és l'exemple perfecte per mostrar que P2 no està solucionat). Tanmateix, sobre P1 no podem dir res.

Contràriament a l'exemple anterior, aquí sí podem veure com P3 no ha desaparegut, i és que part de les branques de la L1, L5, L9S i FM és visitada per l' A^*_{bcn} . El motiu és el mateix pel qual l' A^*_{geo} visita nodes al voltant de la Source en direcció contrària al Target explicat a l'apartat 4.7.4. Si anem a les iteracions concretes, veiem que es deu al baix valor que dona h_{geo} . Tanmateix, la noció de les regions que aporta l' h_{bcn} fa que no es visitin tantes parades com fa l' A^*_{geo} o el Dijkstra. També cal observar que el P2 persisteix però s'ha atenuat.

Si ens fixem en les entrades de la branca d'origen, podem veure que han sigut visitades per l' A^*_{bcn} . Això inicialment pot semblar incoherent amb el que hem comentat més amunt de que el P4 s'havia solucionat pels trams on es recorre exclusivament una branca. No obstant, aquest fet té una explicació ben interessant. Si busquem les iteracions concretes al seu informe podem veure com primer es fan 10 iteracions perfectes extraient els 10 primer vèrtexs del camí mínim fins a l'andana pont. No és fins la iteració 17 que es comencen a explorar aquestes entrades. De fet, s'explora abans una entrada de la parada L10S – Ciutat de la Justícia (a dos parades del pont però més propera al Target en línia recta) que no pas una de la L10S – Provençana (a una parada del pont), i això dona una primera pista sobre l'origen d'aquest fenomen. Una vegada s'arriba al centre, es comença a tenir en compte de nou l' h_{geo} . La seva baixa cota inferior ja comentada prèviament provoca, doncs, aquest succés.

Finalment, obtenim les pitjors mètriques de A^*_{bcn} als casos CB i BC. De totes maneres, anem a analitzar com millora l'algoritme respecte A^*_{geo} . Per això prendrem la mateixa ruta d'exemple mostrada a l'apartat 4.7.4, però ara afegint l' A^*_{bcn} .

A la Figura 14 podem veure com l' A^*_{bcn} ja no visita branques que no li corresponen. Tot i això, els exemples anteriors mostren que el problema P3 no desapareix completament. P1 i P2 es mantenen però s'ha atenuat la seva gravetat. Respecte el P4, veiem que no es dona a la branca de destí (motiu explicat en aquest mateix apartat), i si ens fixem prou, podem veure com a la part del centre també es visiten menys entrades innecessàries.

En resum, l' A^*_{bcn} millora l' A^*_{geo} perquè incorpora dues característiques estructurals pròpies de la xarxa de metro de Barcelona: la distinció entre regions i la penalització de les entrades. Aquesta informació permet reduir l'impacte dels problemes P1, P2 i P3 i eliminar completament el P4 quan el recorregut transcorre íntegrament per una mateixa branca. En la resta de casos, el problema P4 també disminueix de manera apreciable gràcies al cost d'entrada.

4.9 Graf de treball reduït

4.9.1 Motivació i construcció

Si ens remuntem a la Secció 4.7.1, recordem que gairebé tres quartes parts dels vèrtexs del graf corresponen a entrades i que aquestes només intervenen al principi i al final d'un trajecte. També acabem de veure que l' A^*_{bcn} aconsegueix reduir el nombre d'iteracions dedicades a explorar aquestes entrades gràcies al cost d'entrada c_{ent} , però només fins a un cert límit. Si augmentéssim encara més aquesta penalització l'heurística deixaria de ser admissible.

Per tant, ens trobem davant d'una limitació que no sembla possible superar modificant únicament la funció heurística.

Això suggereix un canvi de perspectiva. En lloc d'intentar transmetre als algorismes que les entrades només s'han de visitar al començament i al final del recorregut, podem construir directament un graf on aquestes entrades intermèdies ja no existeixin.

Aquesta idea és possible gràcies als conjunts P_f i P_t introduïts a la Definició 4.8. Donades una Source i un Target corresponents a dues entrades, aquests conjunts permeten determinar les andanes des de les quals realment començarà i acabarà el trajecte. Per tant, l'algoritme de camins mínims es pot executar sobre un graf obtingut a partir de filtrar les arestes de tipus PW de `weights.txt`.

D'aquesta manera, els algorismes ja no tenen la possibilitat d'explorar entrades intermèdies, de manera que el problema P4 identificat a la Secció 4.7.4 queda resolt completament.

La implementació és una extensió natural de la utilitzada per l' h_{bcn} . En primer lloc, es calculen els conjunts P_f i P_t . Si algun dels dos és buit, es conclou que no existeix cap camí vàlid. En cas contrari, s'executa l'algoritme escollit sobre el graf sense arestes PW per a totes les combinacions $(p, q) \in P_f \times P_t$, i es conserva la ruta de pes mínim. Finalment, el camí obtingut es transforma novament en un recorregut complet afegint-hi les entrades inicial i final.

Fins ara totes les millores introduïdes s'havien centrat en el disseny de la funció heurística. Amb aquesta proposta, però, el canvi no recau sobre l'heurística, sinó sobre el graf sobre el qual treballen els algorismes. Es tracta, per tant, d'una estratègia complementària a les anteriors, ja que modifica l'espai de cerca abans fins i tot d'iniciar l'execució dels algorismes.

4.9.2 Avaluació experimental i anàlisi

De nou, repetim exactament la mateixa metodologia experimental emprada a les Seccions 4.5, 4.6.2, 4.7.4 i 4.8.2. Obtenim, doncs, el gràfic de la Figura 5, on podem observar com ha millorat cada algoritme en cada cas pel parell ordenat de Source i Target $(\xi, \gamma) \in V \times V$ al treballar en la nova versió del graf.

A primera vista podem veure com el rendiment global de l' A^*_{bcn} s'ha duplicat, donant-nos ara una mètrica encara lluny de la ideal, però significativament superior a les obtingudes fins ara.

De fet, podem veure com els altres dos algorismes també han millorat globalment i a cada cas mantenint la dominància en ordre de A^*_{cheat} , A^*_{bcn} , A^*_{geo} i Dijkstra donada per la cadena de desigualtats (2). Això es deu a que aquesta nova estratègia soluciona el P4 per complet, i aquest estava present en tots els algorismes. És a dir, com aquest canvi afecta a la base des d'on treballen els algorismes, tots han millorat notòriament.

Pel tipus de canvi fet, els altres tres problemes encara persisteixen. És per això que l'heurística h_{bcn} encara és necessària (no els soluciona però sí redueix el seu impacte). Una evidència d'això són les mètriques globals del Dijkstra (0.21) i de l' A^*_{bcn} (0.39).

Observant el cas CC, veiem que el rendiment de A^*_{bcn} i A^*_{geo} és exactament el mateix. Això es deu al fet que el què les diferenciava (el cost d'entrada) ja no té cap efecte. De fet, ho podem veure de manera més visual comparant la Figura 11, comentada a la Secció 4.8.2, amb la Figura 15. Abans, els únics nodes que visitava A^*_{geo} i A^*_{bcn} no, eren les entrades mostrades en verd. En canvi, ara ja no veiem res en aquest color, però a més, ja no veiem visitada cap entrada en general. Cal remarcar que això últim passa ara amb totes les rutes.

Algorithm performance by graph region: full graph vs graph without entrances

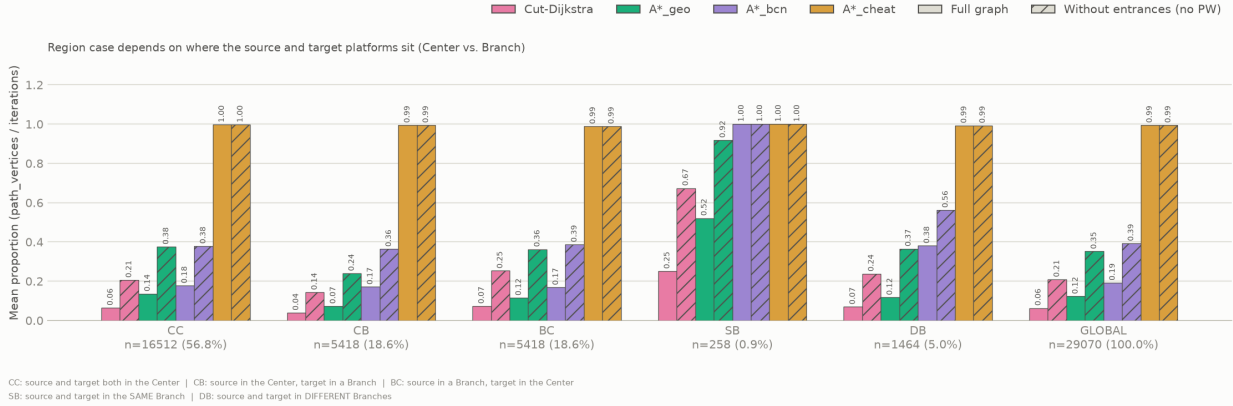


Figura 5: endiment mitjà de cerca del Dijkstra, l'A*_{geo}, l'A*_{bcn} i l'A*_{cheat} segons el cas $(\xi, \gamma) \in V \times V$ amb la versió estàndard del graf i la de treball reduïda. Figura generada amb `algorithms_comparison.py`.

5 Conclusions

L'objectiu d'aquest treball es dividia en dues parts. En primer lloc, construir un model de la xarxa de metro de Barcelona que permetés determinar el recorregut més ràpid entre dos accessos qualssevol mitjançant algoritmes de camí mínim. En segon lloc, estudiar diferents estratègies per reduir el nombre d'iteracions necessàries per obtenir aquest recorregut, mantenint la garantia d'obtenir sempre el camí mínim.

Tots dos objectius s'han assolit satisfactòriament. S'ha construït un model complet de la xarxa a partir de dades GTFS oficials, s'han implementat diversos algoritmes de cerca i s'han desenvolupat i avaluat diferents estratègies d'optimització que permeten reduir significativament el cost de la cerca.

La primera aportació del treball ha estat la construcció d'un model de graf adaptat específicament a la xarxa de metro de Barcelona. A partir del marc teòric i de les dades GTFS oficials, s'han formulat les hipòtesis necessàries, s'ha desenvolupat íntegrament el pipeline de processament de dades i s'han justificat experimentalment les principals decisions de modelatge, com ara la definició dels pesos, la representació dirigida del graf o el tractament de les andanes compartides.

El desenvolupament d'aquest model ha requerit sis etapes de transformació de dades, diversos mecanismes de validació i diferents anàlisis empíriques que han permès garantir la consistència del graf abans d'aplicar-hi els algoritmes de camí mínim.

L'anàlisi experimental mostra que, malgrat la variabilitat inherent dels temps de viatge, representar els pesos SW mitjançant la mitjana dels temps observats constitueix una aproximació prou fidel per a la immensa majoria de la xarxa. Això justifica l'ús de pesos estàtics i permet simplificar notablement tant la construcció del graf com l'aplicació dels algoritmes de camí mínim.

Amb el graf construït, el problema del camí mínim es resol mitjançant l'algoritme Dijkstra, que garanteix sempre l'obtenció del recorregut òptim entre dos accessos qualssevol de la

xarxa.

Tanmateix, l'avaluació experimental de l'algoritme clàssic posa de manifest que, malgrat obtenir sempre la solució òptima, només aproximadament un 6% dels vèrtexs visitats formen part del camí mínim final. Aquest resultat evidencia que existeix un ampli marge de millora i motiva la segona part del treball, centrada a reduir el nombre d'iteracions necessàries per obtenir el mateix recorregut òptim.

La primera aproximació ha consistit en incorporar una heurística geomètrica basada en la distància en línia recta, la qual presenta una millora apreciable.

A continuació, s'ha analitzat la topologia pròpia de la xarxa de metro de Barcelona, identificant característiques estructurals que no apareixen en les heurístiques genèriques. Aquest estudi ha conduït al desenvolupament d'una nova heurística específica per a aquesta xarxa i, posteriorment, a una estratègia basada en la reducció del graf de treball abans de l'execució dels algoritmes.

En conjunt, aquestes aportacions permeten passar d'obtenir aproximadament 6 vèrtexs del camí mínim cada 100 iteracions a obtenir-ne prop de 39, multiplicant l'eficiència del procés aproximadament per un factor de 6.5.

Una primera conclusió és que la resolució eficient del problema del camí mínim no depèn exclusivament de l'algoritme utilitzat, sinó també de la qualitat del model sobre el qual aquest opera. Això reforça la idea que, en la modelització matemàtica, no es tracta de reproduir la realitat amb el màxim nivell de detall possible, sinó de trobar les simplificacions adequades perquè les regularitats generals del sistema continuïn quedant representades. De fet, els resultats obtinguts mostren que són aquestes simplificacions les que proporcionen la millor representativitat global de la xarxa.

Una segona conclusió és que, una vegada construït un bon model, incorporar coneixement específic del problema permet obtenir millores molt superiors a les que proporcionen les aproximacions completament genèriques. En particular, dividir el problema en casos segons les característiques topològiques de la xarxa i estudiar cadascun d'ells ha permès entendre l'origen de les ineficiències dels algoritmes i plantejar solucions adaptades a cada situació. Tant la definició de la heurística h_{bcn} com la reducció del graf de treball aprofiten aquestes peculiaritats de la xarxa de metro de Barcelona, absents en les heurístiques generals. Els resultats experimentals confirmen que aquesta aproximació millora substancialment el rendiment respecte de les aproximacions genèriques i de la solució inicial basada únicament en Dijkstra.

Finalment, aquest treball presenta una solució completa, reproduïble i escalable que cobreix tot el procés, des de les dades GTFS originals fins a l'obtenció eficient del recorregut òptim entre dos accessos qualssevol de la xarxa. El desenvolupament ha requerit combinar coneixements de teoria de grafs, algoritmes, modelització de dades, validació experimental i anàlisi computacional, mostrant que la resolució d'un problema real requereix tant un bon model matemàtic com una interpretació acurada de les dades sobre les quals aquest es construeix.

El model construït es basa en diverses simplificacions. En particular, totes les arestes entre una andana i una entrada comparteixen el mateix pes, cada andana es representa amb un únic vèrtex (sense distingir els dos sentits de circulació) i no es modelen ni els temps d'espera dels trens ni la congestió de passatgers. Aquestes simplificacions poden modificar el cost estimat

dels recorreguts i, en alguns casos, també el camí òptim obtingut. Tanmateix, quan això succeeix, les diferents alternatives acostumen a presentar temps de recorregut molt similars, de manera que totes constitueixen aproximacions raonables al recorregut més ràpid. A més, els algoritmes implementats retornen una única solució òptima, tot i que en poden existir diverses.

Les principals línies futures passen per incorporar temps d'espera dependents de l'hora i del dia, ampliar el model a altres mitjans de transport, estudiar criteris d'optimització alternatius (com minimitzar transbords) i desenvolupar heurístiques més precises que continuïn reduint el nombre d'iteracions.

Referències

- [1] Lluís Alsedà. *Shortest paths algorithms in weighted graphs*. Version 3.3. Modified by Albert Ruiz. Departament de Matemàtiques, Universitat Autònoma de Barcelona. Abr. de 2024. URL: <https://mat.uab.cat/~alseda/MasterOpt/RoutingInGraphs.pdf> (cons. 22-07-2026).
- [2] Bernat Borràs. *El metro de Barcelona – Línia L3 – Liceu*. Trenscat. URL: https://www.trenscat.com/tmb/03liceu_ct.html (cons. 22-07-2026).
- [3] Ministeri de Transports i Mobilitat Sostenible. *Punt Nacional d'Accés a les Dades del Transport*. 2026. URL: <https://nap.transportes.gob.es/> (cons. 14-07-2026).
- [4] MobilityData. *GTFS Schedule Reference*. 2025. URL: <https://gtfs.org/documentation/schedule/reference/> (cons. 08-07-2026).
- [5] John P. Snyder. *Map Projections—A Working Manual*. Vol. 1395. U.S. Geological Survey Professional Paper. Washington, D.C.: United States Government Printing Office, 1987.
- [6] TOT Barcelona. *L'única estació de metro de Barcelona on cal sortir al carrer per canviar de sentit*. 2024. URL: <https://www.totbarcelona.cat/mobilitat/unica-estacio-metro-barcelona-sortir-carrer-canviar-sentit-427931/> (cons. 22-07-2026).
- [7] Transports Metropolitans de Barcelona (TMB). *Mapa del metro i bus de Barcelona 2026*. Transports Metropolitans de Barcelona. 2026. URL: <https://www.tmb.cat/ca/transport-barcelona/mapa-metro-bus> (cons. 22-07-2026).

A Pseudocodis d'algorithmes

Algorithme 1 Dijkstra

```
1: procedure DIJKSTRA( $G, \xi$ ) ▷ iterations = 0
2:    $Pq \leftarrow \emptyset$ 
3:    $\text{expanded}[v] \leftarrow \text{false}, \forall v \in V$ 
4:    $\text{dist}[v] \leftarrow \infty, \forall v \in V$ 
5:    $\text{parent}[v] \leftarrow \text{None}, \forall v \in V$ 
6:    $\text{dist}[\xi] \leftarrow 0$ 
7:    $\text{parent}[\xi] \leftarrow \text{None}$ 
8:    $Pq.ADDWITHPRIORITY(\xi, \text{dist}[\xi])$ 
9:   while  $Pq \neq \emptyset$  do ▷ Main loop starts (iterations += 1)
10:     $\text{node} \leftarrow Pq.EXTRACTMIN$ 
11:     $\text{expanded}[\text{node}] \leftarrow \text{true}$ 
12:    for all  $\text{adj} \in \text{Adj}(\text{node})$  do
13:      if not  $\text{expanded}[\text{adj}]$  then
14:         $\text{dist}_{\text{aux}} \leftarrow \text{dist}[\text{node}] + \omega(\text{node}, \text{adj})$ 
15:        if  $\text{dist}[\text{adj}] > \text{dist}_{\text{aux}}$  then
16:          if  $\text{dist}[\text{adj}] = \infty$  then
17:             $Pq.ADDWITHPRIORITY(\text{adj}, \text{dist}_{\text{aux}})$ 
18:          else
19:             $Pq.DECREASEPRIORITY(\text{adj}, \text{dist}_{\text{aux}})$ 
20:          end if
21:           $\text{dist}[\text{adj}] \leftarrow \text{dist}_{\text{aux}}$ 
22:           $\text{parent}[\text{adj}] \leftarrow \text{node}$ 
23:        end if
24:      end if
25:    end for
26:  end while ▷ Main loop ends
27:  return  $\text{dist}, \text{parent}$ 
28: end procedure
```

Algorithme 2 A^*

```
1: procedure  $A^*(G, \xi, \gamma, h)$  ▷ iterations = 0
2:    $\text{Open} \leftarrow \emptyset$ 
3:    $\text{parent}[v] \leftarrow \text{None}, \forall v \in V$ 
4:    $g[v] \leftarrow \infty, \forall v \in V$ 
5:    $g[\xi] \leftarrow 0$ 
6:    $\text{parent}[\xi] \leftarrow \text{None}$ 
7:    $\text{Open.ADDWITHPRIORITY}(\xi, g[\xi], h(\xi, \gamma))$ 
8:   while  $\text{Open} \neq \emptyset$  do ▷ Main loop starts (iterations+ = 1)
9:      $\text{node} \leftarrow \text{Open.EXTRACTMIN}$ 
10:    if  $\text{node} = \gamma$  then
11:      return  $g, \text{parent}$ 
12:    end if
13:    for all  $\text{adj} \in \text{Adj}(\text{node})$  do
14:       $g_{\text{aux}} \leftarrow g[\text{node}] + \omega(\text{node}, \text{adj})$ 
15:      if  $g_{\text{aux}} < g[\text{adj}]$  then
16:         $\text{parent}[\text{adj}] \leftarrow \text{node}$ 
17:         $g[\text{adj}] \leftarrow g_{\text{aux}}$ 
18:        if not  $\text{Open.BELONGSTO}(\text{adj})$  then
19:           $\text{Open.ADDWITHPRIORITY}(\text{adj}, g[\text{adj}], h(\text{adj}, \gamma))$ 
20:        else
21:           $\text{Open.DECREASEPRIORITY}(\text{adj}, g[\text{adj}], h(\text{adj}, \gamma))$ 
22:        end if
23:      end if
24:    end for
25:  end while ▷ Main loop ends
26:  return failure
27: end procedure
```

B Construcció del graf

B.1 Pipeline de validació i processament de dades

Aquest annex recull el procés complet de construcció del graf descrit a la Secció 3, des de les dades GTFS originals fins al graf ponderat final. La implementació corresponent es troba al directori `data_validation` del repositori.

El procés no consisteix únicament en una successió de transformacions. En aquells casos en què ha estat necessari modificar les dades, primer s'ha implementat una comprovació que detecta el problema a corregir. Després d'aplicar la correcció, aquesta mateixa comprovació es torna a executar per verificar que el problema ha desaparegut. Aquest esquema de detecció, correcció i verificació es repeteix en tres de les sis etapes principals de la construcció del graf.

A més, el procés incorpora quatre comprovacions addicionals que agrupen dotze comprovacions de consistència i tres estudis que han servit de suport a les decisions de modelització descrites al llarg de la memòria. En conjunt, aquesta fase de validació i anàlisi ha representat una part molt important del treball desenvolupat.

B.1.1 Etapes del pipeline principal

El pipeline està format per sis etapes consecutives. Cada etapa genera un nou conjunt de dades, emmagatzemat al directori `data` (`0_raw` $\rightarrow$ `6_weights`), i s'implementa mitjançant un fitxer del directori `data_validation/processing`. La Figura 6 en resumeix l'entrada, la sortida i la transformació principal. A continuació es descriu cadascuna de les etapes amb més detall.

Etapa 1: extracció del metro (`1_subway.py`). Les dades GTFS de TMB inclouen conjuntament la xarxa de metro, la d'autobús i el funicular de Montjuïc. Aquesta primera etapa filtra els fitxers `routes`, `stop_times`, `stops` i `trips`, eliminant totes les files l'identificador de les quals comença per `2.*` (xarxa de bus), i produeix els quatre fitxers homòlegs amb sufix `_subway`. Aquesta etapa és la que fa que les quatre comprovacions independents descrites a l'Annex B.1.3 ja puguin executar-se.

Etapa 2: eliminació de viatges duplicats (`2_duplicated_trips.py`). Aquesta etapa consumeix `trip_ids_to_eliminate.txt`, un fitxer que no genera ella mateixa sinó la comprovació `checks/stop_times/1_duplicate_trips_check.py` (vegeu l'Annex B.1.2). Un viatge es considera duplicat quan la seva seqüència completa de parades (al fitxer `stop_times`), ordenada per `stop_sequence` i amb els seus horaris, coincideix exactament amb la d'un altre `trip_id`; d'aquests grups només se'n conserva un representant. Sobre les dades reals s'han trobat 4085 grups d'aquest tipus (8196 `trip_id` en total). D'aquests 8196, se'n marquen 4111 per eliminar, i el nombre de viatges vàlids passa de 15088 a 10977. El resultat és `stop_times_cleaned.txt` i `trips_cleaned.txt`.

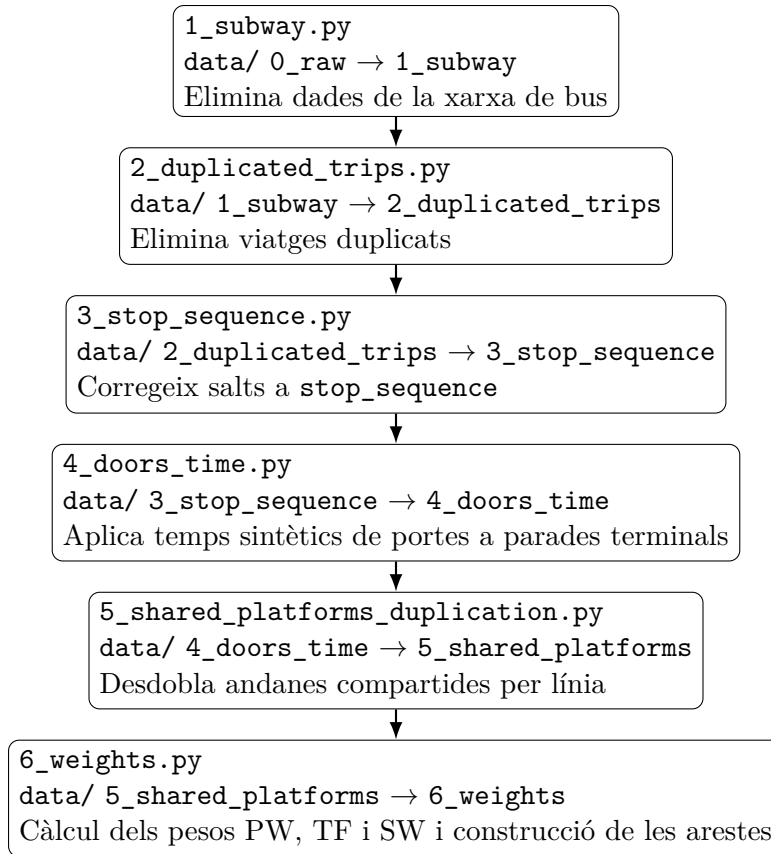


Figura 6: Etapes del pipeline principal

Etapa 3: correcció de `stop_sequence` (`3_stop_sequence.py`). Aquesta etapa corregeix un problema subtil: el camp `stop_sequence` del fitxer `stop_times` numera consecutivament totes les parades d'un viatge (`trip_id`), sense distingir si dues parades consecutives són físicament adjacents o si existeix una parada intermèdia que aquell viatge no efectua.

Corregir aquesta inconsistència és fonamental, ja que la construcció de les arestes del graf es basa en la seqüència consecutiva de les parades. Si no es modifica aquest camp sintèticament, el graf acabaria incorporant arestes entre parelles de vèrtexs de les quals no tenim evidència que estiguin connectades directament a la realitat.

Per fer-ho, el programa utilitza el fitxer `wrong_stop_sequences.txt`, generat per `checks/stop_times/2_canonical_sequence_check.py`, que identifica, per a cada viatge, els punts on dues parades consecutives no són adjacents segons l'ordre canònic de la línia. En cadascun d'aquests punts s'obre un *forat* a la numeració: la parada on es produeix el salt i totes les posteriors dins del mateix viatge incrementen el seu valor de `stop_sequence` en una unitat. Aquests increments són acumulatius; per exemple, una parada situada després de dos punts de ruptura incrementa el seu valor original en dues unitats.

Sobre les dades reals, aquest fenomen només afecta 73 dels 15 088 viatges analitzats, tots ells de la línia L9S. Els salts es concentren en dos trams recurrents, presents en tots dos sentits de circulació: entre Aeroport T2 i Parc Nou, on alguns serveis no efectuen parada a Mas Blau, i entre Fira i Les Moreres, on s'ometen les parades de Mercabarna i Parc Logístic. En

total, el fitxer `wrong_stop_sequences.txt` conté 144 punts de ruptura.

D'aquesta manera, els algoritmes de construcció del graf poden distingir les parades realment adjacents de les que no ho són, sense modificar els horaris d'arribada ni de sortida. El resultat d'aquesta etapa és el fitxer `stop_times_sequence.txt`.

Etapa 4: temps de portes a parades terminals (`4_doors_time.py`). A les parades terminals, les dades GTFS sovint registren el mateix instant per a `arrival_time` i `departure_time`. Aquest fenomen es dona tant a les terminals d'una línia com en aquells viatges que comencen o acaben el recorregut a mitja línia (per exemple, a l'inici o al final de la jornada del servei, per garantir l'abastiment de totes les parades).

Això és un problema perquè els pesos SW es calculen com la mitjana dels temps de recorregut observats entre dues parades consecutives. Si en alguns viatges el temps de portes és nul i en d'altres no, aquesta estimació queda esbiaixada. Per aquest motiu, aquesta correcció s'aplica abans de calcular els pesos SW, garantint que aquests siguin coherents amb el temps real de recorregut.

Aquesta etapa, utilitzant els fitxers `doors.txt` (generat per `checks/stop_times/3_door_times_check.py`) i `trips_cleaned.txt`, reconstrueix sintèticament el temps d'obertura de portes. Concretament, si la parada és la primera del viatge (`stop_sequence` mínim), s'aplica

$$\text{arrival_time} = \text{departure_time} - \text{door_seconds},$$

mentre que si és l'última (`stop_sequence` màxim), s'aplica

$$\text{departure_time} = \text{arrival_time} + \text{door_seconds}.$$

El valor de `door_seconds` per a cada parella (`stop_id`, línia) s'assigna al fitxer `doors.txt` segons tres casos:

- **Parades parcials:** només alguns viatges contenen files amb `arrival_time` i `departure_time` iguals. S'utilitza la mitjana dels temps de portes observats en aquella mateixa parada.
- **Terminals canòniques:** tots els viatges de la línia tenen `arrival_time` i `departure_time` iguals. S'utilitza la mitjana dels temps de portes de tota la línia.
- **Línia FM:** com que les dues úniques parades són terminals i no existeix cap observació directa, s'utilitza la mitjana de totes les altres línies.

Aplicant aquest procediment a les dades reals s'obtenen 63 parelles (`stop_id`, línia) al fitxer `doors.txt`. Com a exemples de terminals canòniques hi trobem les parades L1 – Hospital de Bellvitge i L1 – Fondo, amb 1 407 i 1 405 viatges respectivament amb `arrival_time` i `departure_time` iguals. En tots aquests casos s'assigna la mitjana del temps de portes de la línia corresponent.

En canvi, a les parades parcials la mitjana es calcula únicament a partir dels viatges en què s'observa un temps de portes diferent de zero i s'utilitza posteriorment per corregir els que

presenten `arrival_time` i `departure_time` iguals. Per exemple, a la parada L1 – Santa Eulàlia només 6 dels 1412 viatges registren aquest fenomen; aquests 6 casos es corregeixen utilitzant la mitjana calculada a partir dels 1406 viatges restants (30.21 s, $\sigma = 3.96$ s).

La Taula 7 recull, per a cada línia, la desviació estàndard i la mitjana utilitzada quan cal modificar el temps de portes a les terminals canòniques. Les parades parcials, en canvi, utilitzen la seva pròpia mitjana calculada a partir dels viatges observats.

El resultat d'aquesta etapa és el fitxer `stop_times_doors.txt`.

Línia	μ (s)	σ (s)	n
L1	23.65	6.59	39 475
L2	17.00	6.16	18 910
L3	15.75	3.35	27 238
L4	14.90	2.97	26 158
L5	22.57	4.49	37 597
L9S	26.14	4.47	11 822
L9N	25.03	0.40	6 496
L10S	36.71	13.89	8 123
L10N	25.03	0.59	3 716
L11	36.05	16.18	1 929
FM	-	-	0

Taula 7: Mitjana i desviació estàndard dels temps de portes per línia, excloent les files amb `arrival_time` i `departure_time` iguals. Taula creada a partir de `3_door_times_check.py`.

Etapa 5: desdoblament d'andanes compartides (`5_shared_platforms_duplication.py`). Algunes andanes són compartides per més d'una línia, com passa als trams comuns de les línies L9S/L10S i L9N/L10N (vegeu la Taula 2). Aquesta etapa desdobla cada `stop_id` compartit (per exemple, 1.930) en un identificador diferent per a cada línia (1.9300, 1.9301, ...).

La detecció de les andanes compartides és completament automàtica. Es dedueix a partir de `subway_route_names_stop_ids` (diccionari de `subway_reference/subway_lines.py`), de manera que no és necessari codificar manualment cap nom de línia i qualsevol nova andana compartida seria detectada automàticament.

El tractament depèn del fitxer que es modifica:

- Als fitxers `stops`, `pathways` i `transfers`, cada fila es duplica, generant una còpia per a cadascuna de les línies que utilitzen aquella andana amb un `stop_id`. Per exemple, l'identificador 1.930 passaria a ser 1.9300, 1.9301, ...
- A més, als fitxers `pathways` i `transfers` s'afegeix una aresta de correspondència entre cada parell de nous identificadors d'una mateixa andana, amb pes igual al `min_transfer_time` mínim observat a `transfers`.

- Al fitxer `stop_times`, en canvi, les files no es dupliquen. Simplement es substitueix el `stop_id` original pel nou identificador corresponent a la línia del viatge, ja que cada `trip_id` pertany a una única línia.

Aquesta etapa genera les versions `_shared.txt` dels quatre fitxers anteriors i crea també `equivalences.txt`, que emmagatzema la correspondència

`(original_stop_id, line, new_stop_id)`

de totes les andanes desdoblades.

La necessitat d'aquest desdoblament queda justificada empíricament a l'anàlisi `analysis/shared_platforms.py` (Annex B.1.4).

Etapa 6: construcció de les arestes ponderades (`6_weights.py`). Aquesta és la darrera etapa del pipeline i genera els dos fitxers finals: `subway_weights.txt` i `weights.txt`. En primer lloc, es construeixen les arestes SW (andana-andana). Per a cada parell dirigit de parades consecutives (`stop_a`, `stop_b`) segons el camp `stop_sequence`, es calcula el pes com la mitjana dels temps de recorregut observats,

`arrival_b - arrival_a,`

agrupant els viatges per (línia, `direction_id`). La mitjana es calcula mitjançant la funció `round_half_up_mean`, que treballa amb aritmètica d'enters i evita els problemes d'arrodoniment que introdueix la combinació de `statistics.mean` i `round()`.

A continuació, aquestes arestes es combinen amb els altres dos tipus del model:

- TF: entre andanes de la mateixa parada, amb pes igual al `min_transfer_time` de `transfers_shared.txt`.
- PW: entre entrades i andanes exclusivament, amb pes igual al `traversal_time` de `pathways_shared.txt`, restringint-se als parells entrada-andana.

El resultat és el fitxer `weights.txt`, que correspon al graf ponderat utilitzat durant la resta del treball.

Aquesta etapa també materialitza les dues decisions justificades a l'Annex B.1.4: considerar els pesos SW dirigits (un per a cada sentit de circulació) i utilitzar un únic pes estàtic, obtingut com la mitjana dels temps observats.

B.1.2 El bucle de detecció, correcció i verificació

Les etapes 2, 3 i 4 no es poden entendre com un simple fitxer de processament aïllat. Cadascuna depèn d'un fitxer de `checks/stop_times/` que primer detecta les files problemàtiques i, un cop aplicada la correcció, es torna a executar amb l'argument `verify`. Aquesta segona execució no modifica cap dada, sinó que actua com una comprovació de sanitat, confirmant que la correcció aplicada a l'etapa de processament corresponent ha eliminat efectivament el problema detectat. La Figura 7 resumeix aquest patró, que es repeteix en les tres etapes.

Concretament:

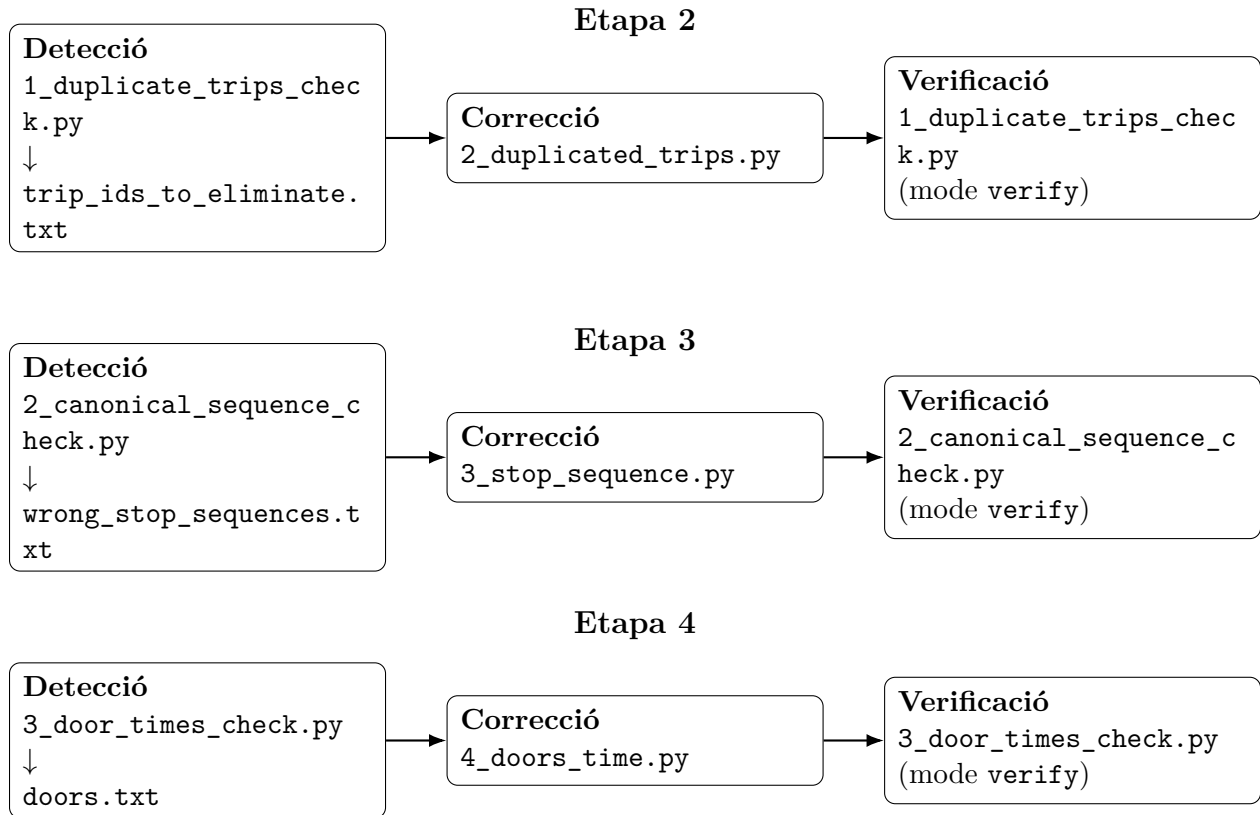


Figura 7: Patró de detecció, correcció i verificació utilitzat a les etapes 2, 3 i 4 del pipeline

- **Duplicats (etapa 2).** `1_duplicate_trips_check.py` construeix, per a cada `trip_id`, una signatura canònica ordenada de les seves files de `stop_times`; quan dues o més signatures coincideixen exactament, tots els `trip_id` implicats menys un es marquen per eliminar a `trip_ids_to_eliminate.txt`. En la detecció s'han trobat 4085 grups de signatures repetides (8196 `trip_id`), dels quals 4111 es marquen per eliminar. Un cop `2_duplicated_trips.py` els ha eliminat correctament, l'execució en mode `verify` torna a calcular les signatures sobre `stop_times_cleaned.txt` com a comprovació de sanitat, i confirma que cap ja no correspon a dos o més `trip_id`.
- **Seqüència canònica (etapa 3).** `2_canonical_sequence_check.py` compara, per a cada viatge, l'ordre de parades observat amb l'ordre canònic de la seva línia, i n'extreu els parells (`seq_a`, `seq_b`) on la parada de seqüència `seq_b` no és, segons la ruta oficial, l'adjacent immediata de la de seqüència `seq_a`. La detecció troba 73 viatges afectats, tots de la línia L9S, amb 144 parells escrits a `wrong_stop_sequences.txt`. Un cop `3_stop_sequence.py` ha obert correctament els forats corresponents, la verificació torna a comparar contra l'ordre canònic sobre `stop_times_sequence.txt` com a comprovació de sanitat, i confirma que els 144 parells problemàtics ja mostren una diferència de seqüència no consecutiva.
- **Temps de portes (etapa 4).** `3_door_times_check.py` recorre `stop_times_sequence.txt` i, per a cada parella (línia, `stop_id`), compta quants dels seus registres

tenen `arrival_time == departure_time`, distingint si l'ocurrència és a una parada terminal (`stop_sequence` mínim o màxim del viatge) o parcial; amb això calcula les mitjanes de temps de portes que s'escriuen a `doors.txt` (63 files, vegeu la Taula 7 per a les mitjanes per línia). Un cop `4_doors_time.py` ha reconstruït correctament els temps sintètics, la verificació torna a comprovar sobre `stop_times_doors.txt`, com a comprovació de sanitat, que cap parada ja no té `arrival_time == departure_time`.

Aquest bucle explica per què el directori `checks/` i el directori `processing/` estan organitzats per tipus de fitxer i no per ordre d'execució: cada fitxer de `checks/stop_times/` apareix, de fet, dues vegades a l'ordre real d'execució del pipeline (un cop en mode detecció i un cop en mode `verify`), intercalat entre els fitxers de `processing/`.

B.1.3 Comprovacions independents

A banda de les comprovacions que formen part del bucle de detecció, correcció i verificació, el projecte inclou quatre scripts addicionals de comprovació que agrupen dotze comprovacions de consistència. Aquests es poden executar en qualsevol moment després de l'etapa 1, ja que no depenen de l'ordre del pipeline ni generen fitxers consumits posteriorment; validen i proporcionen informació sobre diferents propietats de les dades i del model construït, que serveixen de base per a algunes de les decisions preses durant el desenvolupament.

Connexions entre fitxers (`file_connection_checks.py`). Aquesta comprovació verifica la de les referències entre els diferents fitxers GTFS. En concret, es comprova el següent.

- Tots els `stop_id` presents a `pathways`, `transfers` i `stop_times` existeixen a `stops`.
- Els `route_id` de `trips` coincideixen amb els de `routes`.
- Els `trip_id` de `stop_times` coincideixen amb els de `trips`.

Per tant, no s'ha detectat cap inconsistència.

Connectivitat de `pathways` i andanes (`pathways_checks.py`). Aquesta comprovació valida la coherència estructural dels fitxers `pathways` i `transfers`. En concret, es verifica el següent.

- Els parells andana–andana presents a `pathways` i `transfers` coincideixen exactament (30 parells als dos fitxers). És a dir, `pathways` inclou tots els recorreguts d'andana a andana i els d'entrada a andana (o a l'inrevés). Per tant, el fitxer `transfers` està inclòs dins del de `pathways`.
- Dels 1063 recorreguts que conté `pathways`, només n'hi ha un que no disposa de la connexió inversa: existeix `PW.1.120_E.12001` però no `PW.E.12001_1.120`. Aquest cas es tracta de l'entrada Alcolea (escala mecànica) de la parada L1 – Plaça de Sants. Aquesta asimetria s'ha confirmat mitjançant una inspecció presencial. A la Figura 8 es mostra com l'escala mecànica només opera en sentit de sortida cap a l'exterior, sense cap tram equivalent en sentit invers.

- Descartant aquest únic cas, el camp `traversal_time` és idèntic als dos sentits per als 531 parells comparables.
- Tots els valors de `traversal_time` són múltiples de 15 segons i, en els 1 003 pathways entrada-andana, prenen sempre el valor 60.

També es comprova la connectivitat de la xarxa.

- Cadascuna de les 502 entrades està connectada a exactament una andana. És a dir, no hi ha cap entrada aïllada o incomunicada.
- De les 165 andanes, 158 disposen d'almenys un accés directe des d'una entrada. Les 7 restants (Taula 8) hi queden connectades mitjançant una aresta TF amb una altra andana de la mateixa parada, que sí està connectada amb una entrada. És a dir, no hi ha cap andana aïllada o incomunicada.
- Les 22 parades amb més d'una andana disposen d'una aresta TF entre cada parell d'andanes.

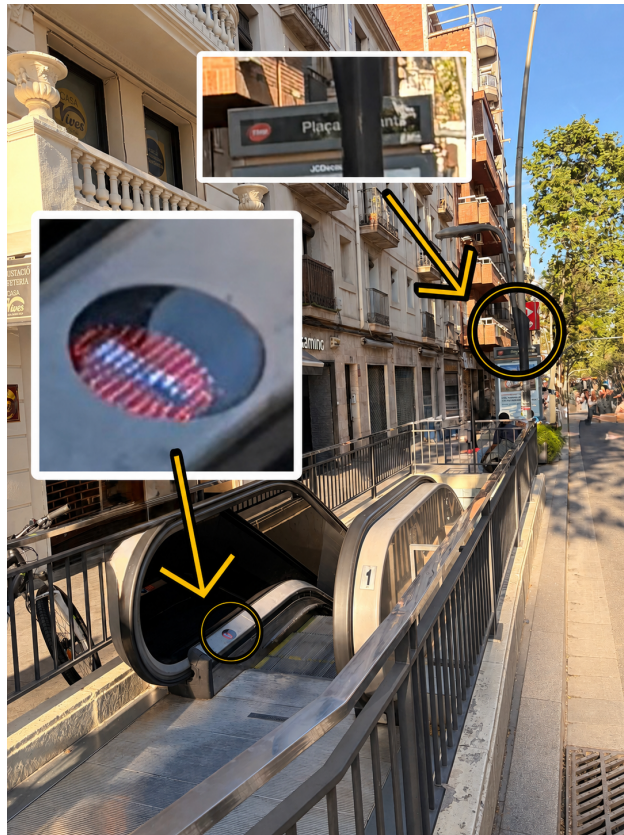


Figura 8: Entrada Alcolea (escala mecànica) de l'estació L1 – Plaça de Sants. Font: elaboració pròpia.

Andana sense entrada directa	Andana de connexió
L2 – La Pau	L4 – La Pau
L3 – Paral·lel	L2 – Paral·lel
L9S–L10S – Torrassa	L1 – Torrassa
L9N–L10N – La Sagrera	L1 – La Sagrera L5 – La Sagrera
L9N – Fondo	L1 – Fondo
L11 – Trinitat Nova	L3 – Trinitat Nova L4 – Trinitat Nova
FM – Paral·lel	L2 – Paral·lel

Taula 8: Andanes sense accés directe a cap entrada i andanes de la mateixa parada amb accés directe a almenys una entrada a través de les quals queden connectades. Taula creada a partir de `pathways_checks.py`.

Aparellament de direcció i destinació (`trips_checks.py`). Per a cadascuna de les 11 línies (15 088 viatges en total), cada ruta presenta exactament dos valors de `trip_headsign`, que corresponen de manera unívoca als dos valors possibles de `direction_id` (Taula 9). No s’ha detectat cap ruta amb una correspondència ambigua o inconsistent.

Línia	<code>trip_headsign (direction_id 0)</code>	<code>trip_headsign (direction_id 1)</code>
L1	Fondo	Hospital de Bellvitge
L2	Badalona Pompeu Fabra	Paral·lel
L3	Trinitat Nova	Zona Universitària
L4	Trinitat Nova	La Pau
L5	Vall d’Hebron	Cornellà Centre
L9S	Zona Universitària	Aeroport T1
L9N	Can Zam	La Sagrera
L10S	Collblanc	ZAL Riu Vell
L10N	Gorg	La Sagrera
L11	Can Cuiàs	Trinitat Nova
FM	Parc de Montjuïc	Paral·lel

Taula 9: Aparellament de `trip_headsign` i `direction_id` per línia. Taula creada a partir de `trips_checks.py`.

Increment de `stop_sequence` (`stop_times/sequence_increment_check.py`). Pels 15 088 viatges, el camp `stop_sequence` incrementa sempre d’una unitat en una unitat entre para-des consecutives d’un mateix viatge. Aquest resultat és molt important perquè ens basarem en la seqüència consecutiva d’aquest camp per determinar les arestes existents al graf.

B.1.4 Anàlisis de suport a les decisions de disseny

El directori `analysis/` conté tres scripts que no formen part del pipeline de transformació de dades, sinó que es van utilitzar durant el desenvolupament del treball per justificar diverses decisions de disseny adoptades en la construcció del graf.

- **`shared_platforms.py`**. Analitza el comportament de les andanes compartides entre diverses línies i justifica el seu desdoblament en un node per línia (Secció 3.3.1).
- **`directional_asymmetry.py`**. Estudia les diferències entre els temps de viatge en ambdós sentits d'una mateixa connexió i justifica la utilització d'un graf dirigit (Secció 3.3.1).
- **`edge_weight_validation.py`**. Analitza la variabilitat temporal dels temps de viatge observats i avalua la idoneïtat d'utilitzar un únic pes estàtic per aresta (Secció 3.4).

Els resultats obtinguts amb aquests scripts i la discussió de les decisions que en deriven ja s'han presentat a les seccions corresponents de la memòria.

B.1.5 Consideracions finals

Convé remarcar que, tot i que la construcció del graf final, descrita a la Secció 3, culmina amb la generació del fitxer `weights.txt`, arribar-hi ha requerit sis etapes de transformació, tres bucles de detecció, correcció i verificació, quatre scripts de comprovació que agrupen dotze comprovacions de consistència, i tres anàlisis empíriques de suport a les decisions de disseny. La major part del temps invertit en aquest treball ha anat, precisament, a garantir que cada decisió de modelatge (la direccionalitat del graf, el desdoblament de les andanes compartides o l'ús d'un pes estàtic per a les arestes SW) estigués justificada per les dades i no fos una simplificació arbitrària.

B.2 Sortides de programes

Llistat 1: Sortida retallada del programa `directional_asymmetry.py`

```
$ python directional_asymmetry.py

Top pairs by absolute directional difference:
1. 1.915 (L10S-Torrassa) -> 1.916 (L10S-Collblanc)
   dir0 1:40 (100.00s) cnt=465 stdev=0.00s
   dir1 2:50 (169.58s) cnt=462 stdev=8.48s
   diff 1:10 (69.58s)

2. 1.915 (L9S-Torrassa) -> 1.916 (L9S-Collblanc)
   dir0 1:40 (100.00s) cnt=469 stdev=0.00s
   dir1 2:47 (166.75s) cnt=458 stdev=11.18s
   diff 1:07 (66.75s)
```

```
...
```

Llistat 2: Sortida retallada del programa `shared_platforms.py`

```
$ python shared_platforms.py

-----L9S/L10S-----
Direction_id=0---

...

1.915 (Torrassa) -> 1.916 (Collblanc)
  L9S 1:40 (100.00s) cnt=469 stdev=0.00s
  L10S 1:40 (100.00s) cnt=465 stdev=0.00s
  diff 0:00 (0.00s)

Direction_id=1---

...

1.916 (Collblanc) -> 1.915 (Torrassa)
  L9S 2:47 (166.75s) cnt=458 stdev=11.18s
  L10S 2:50 (169.58s) cnt=462 stdev=8.48s
  diff 0:03 (2.82s)

-----L9N/L10N-----

...
```

Llistat 3: Sortida retallada del programa `edge_weight_validation.py`

```
$ python edge_weight_validation.py

Number of edges with at least one sample: 320

=====
EDGE STABILITY BY COEFFICIENT OF VARIATION
=====

CV = stdev / mean, computed from total travel time.

Stability summary:
  CV < 5% extremely stable: 277 (86.56%)
  5% <= CV < 10% stable: 36 (11.25%)
  10% <= CV < 15% acceptable: 6 (1.88%)
  15% <= CV unstable: 1 (0.31%)
```

```

Edges ranked by total CV (highest first, 15% <= CV): 1 of 320 (0.31%)
1. L11 dir1 1.1138 (Torre Baro | Vallbona) -> 1.1137 (Casa de l'Aigua) [CV
  =19.88%: unstable]
  total mean=2:34 (153.55s) median=2:10 (130.00s) cnt=321 stdev=30.53s cv
    =19.88%
  door 0:52 (52.30s) cnt=321 stdev=31.50s
  sw 1:41 (101.25s) cnt=321 stdev=0.97s
  -> stdev dominated by door

=====
TIME-OF-DAY VARIATION BY HOURLY RANGE
=====

Hourly range = max(hourly mean) - min(hourly mean), normalized by overall total
  mean.

Time-of-day summary:
  hourly range < 5% static weight is excellent: 204 (64.15%)
  5% <= hourly range < 15% static weight still reasonable: 89 (27.99%)
  15% <= hourly range time-dependent routing would help: 25 (7.86%)

Edges ranked by total hourly range percentage (highest first, range >= 15%): 25
  of 318 (7.86%)
1. L10S dir0 1.959 (Provençana) -> 1.9141 (Can Tries | Gornal) [range=34.58s,
  30.80%]
  overall total 1:52 (112.27s) cnt=465 stdev=13.17s
  min hourly mean 01:00, 02:00, 03:00, 04:00 1:25 (85.00s)
  max hourly mean 23:00 2:00 (119.58s)
2. L10S dir0 1.956 (Foc) -> 1.957 (Foneria) [range=34.38s, 29.70%]
  overall total 1:56 (115.78s) cnt=465 stdev=12.46s
  min hourly mean 01:00, 02:00, 03:00 1:30 (90.00s)
  max hourly mean 23:00 2:04 (124.38s)

...

```

C Cerca de camins mínims

C.1 Cas particular de la parada L3 – Liceu

Tal com s'ha introduït a la Secció 3.3.2, la parada L3 – Liceu presenta una particularitat que no queda completament representada en el model del graf.

En total disposa de tres parelles d'entrades (vegeu el Llistat 4), cadascuna de les quals dona accés a un únic sentit de circulació. En canvi, per l'estructura de les dades i el graf, cada vèrtex entrada està connectat amb el vèrtex de l'andana sense tenir en compte el sentit de

la circulació.

Com que aquesta parada no disposa de correspondència amb cap altra línia, aquesta limitació del model del graf només pot afectar aquelles rutes en què el node d'origen o el node de destí sigui una de les seves entrades.

La solució adoptada és comuna a tots els algorismes descrits en la Secció 4.

Concretament, primer es calcula el camí mínim sobre el graf tal com ha estat construït. Un cop obtinguda la ruta, es comprova si l'entrada d'origen o de destí és compatible amb el sentit de circulació en què s'inicia o finalitza el recorregut. En cas contrari, l'entrada es substitueix per la seva entrada parella corresponent.

Quan aquesta correcció és necessària, s'indica explícitament en la sortida generada per l'algoritme. D'aquesta manera, es manté la simplicitat del model sense renunciar a representar correctament aquest cas particular.

Per a més detalls tècnics sobre la implementació, es convida el lector a consultar la funció `apply_liceu_entrance_fix` del fitxer `algorithms_utils.py`.

C.2 Exemple d'una proporció inferior a 1 amb l'heurística ideal

Posem en evidència mitjançant un exemple de per què la proporció presentada a la Secció 4.3 entre el nombre de vèrtexs al camí mínim i el nombre d'iteracions d'un algoritme amb la heurística ideal no sempre és 1.

Considerem el graf format pels vèrtexs S (origen), A , B i T (destí), amb pesos

$$\omega(S, A) = 4, \quad \omega(S, B) = 5, \quad \omega(A, T) = 6, \quad \omega(B, T) = 5,$$

i una heurística ideal donada per

$$h(S, T) = 10, \quad h(A, T) = 6, \quad h(B, T) = 5, \quad h(T, T) = 0.$$

A la primera iteració s'expandeix S . A continuació, la cua de prioritats conté els vèrtexs A i B , tots dos amb

$$f = g + h = 10.$$

Com que tenen la mateixa prioritats, l'ordre d'extracció depèn únicament de la implementació de la cua. Suposem que s'extreu primer B . En aquest moment, la cua passa a contenir A i T , novament amb el mateix valor $f = 10$. Si ara s'extreu A , aquest vèrtex serà explorat tot i no pertànyer al camí mínim que finalment retornarà l'algoritme. A la iteració següent s'expandeix T i s'obté el camí

$$S \rightarrow B \rightarrow T,$$

que és òptim.

En aquest exemple s'han explorat quatre vèrtexs, mentre que el camí mínim només en conté tres. En conseqüència, la proporció anterior és igual a 0.75 malgrat disposar d'una heurística ideal.

C.3 Exemples de nodes explorats

BC case (with the full graph): extracted nodes from Residència sanitària -- L1-Hospital de Bellvitge to Badajoz -- L1-Glòries

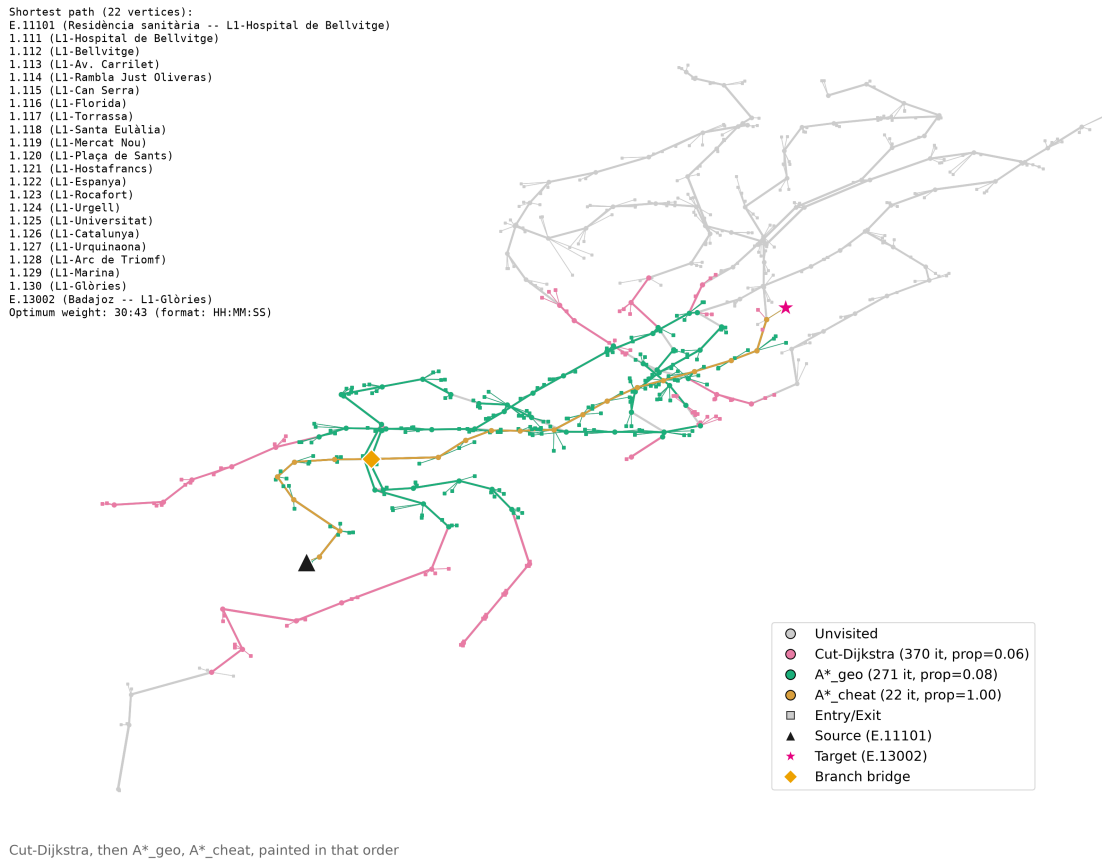


Figura 9: Exemple cas BC de nodes explorats pel Dijkstra, l'A*_{geo} i l'A*_{cheat}, amb origen entrada Residència sanitària de L1 – Hospital de Bellvitge i destí entrada Badajoz de L1 – Glòries. Figura generada amb `extracted_nodes_graph.py`.

CB case (with the full graph): extracted nodes from Badajoz -- L1-Glòries to Residència sanitària -- L1-Hospital de Bellvitge

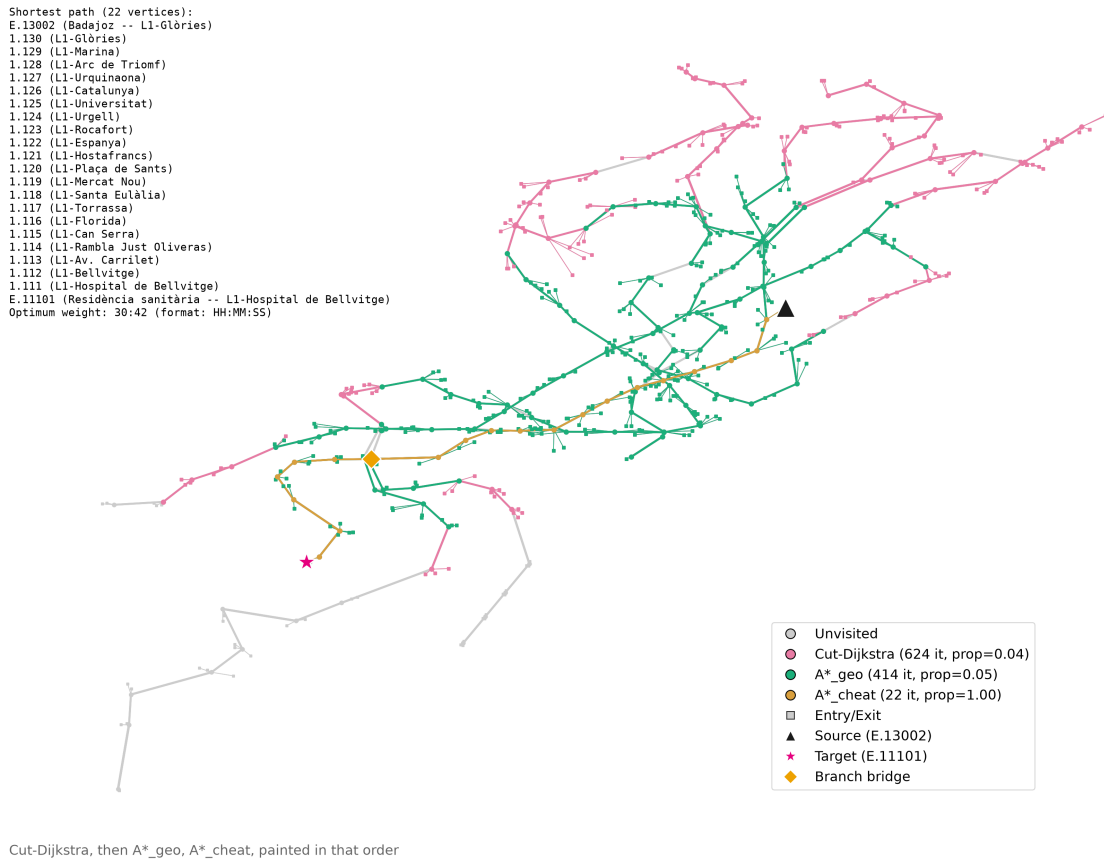


Figura 10: Exemple cas CB de nodes explorats pel Dijkstra, l'A*_geo i l'A*_cheat, amb origen entrada Badajoz de L1 – Glòries i destí entrada Residència sanitària de L1 – Hospital de Bellvitge. Figura generada amb `extracted_nodes_graph.py`.

CC case (with the full graph): extracted nodes from Palau Olímpic Vall d'Hebron -- L3-Montbau to Villarroel -- L5-Hospital Clínic

Shortest path (11 vertices):
E.33403 (Palau Olímpic Vall d'Hebron -- L3-Montbau)
1.334 (L3-Montbau)
1.333 (L3-Vall d'Hebron)
1.332 (L3-Penitents)
1.331 (L3-Vallcarca)
1.330 (L3-Lesseps)
1.329 (L3-Fontana)
1.328 (L3-Diagonal)
1.521 (L5-Diagonal)
1.520 (L5-Hospital Clínic)
E.52001 (Villarroel -- L5-Hospital Clínic)
Optimum weight: 16:18 (format: HH:MM:SS)

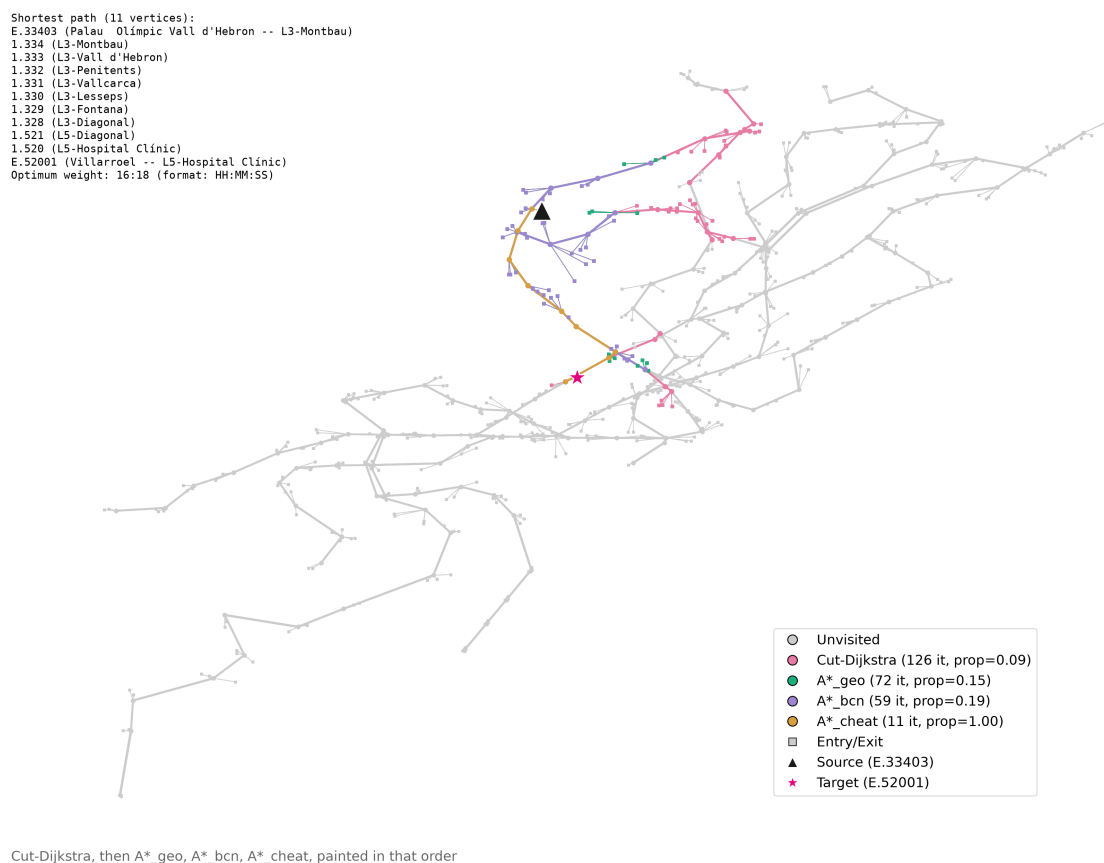


Figura 11: Exemple cas CC de nodes explorats pel Dijkstra, l'A*_{geo}, l'A*_{bcn} i l'A*_{cheat}, amb origen entrada Palau Olímpic Vall d'Hebron de L3 – Montbau i destí entrada Villarroel de L5 – Hospital Clínic. Figura generada amb `extracted_nodes_graph.py`.

DB case (with the full graph): extracted nodes from Aeroport T1 – L9S-Aeroport T1 to ZAL – L10S-ZAL | Riu Vell

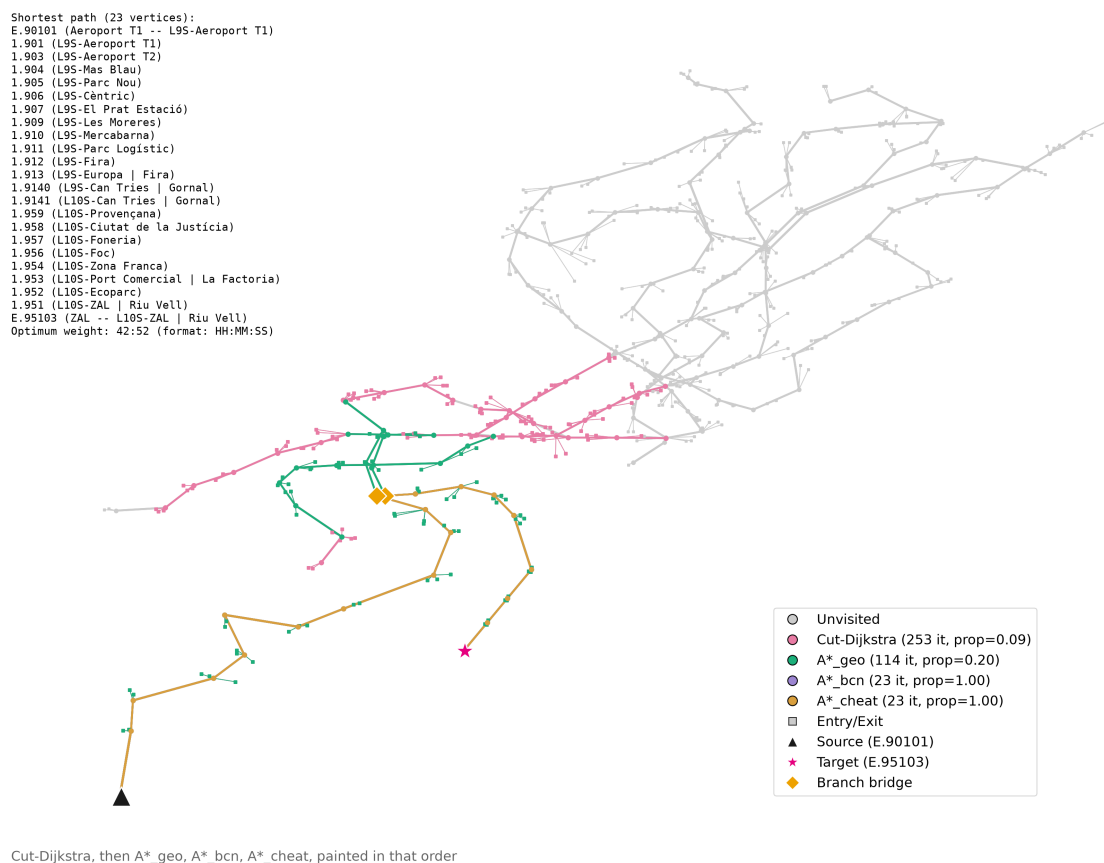


Figura 12: Exemple cas DB de nodes explorats pel Dijkstra, l' A^*_{geo} , l' A^*_{bcn} i l' A^*_{cheat} , amb origen entrada Aeroport T1 de L9S – Aeroport T1 i destí entrada ZAL de L10S – ZAL | Riu Vell. Figura generada amb `extracted_nodes_graph.py`.

DB case (with the full graph): extracted nodes from ZAL -- L10S-ZAL | Riu Vell to Francesc Macià -- L9N-Can Zam

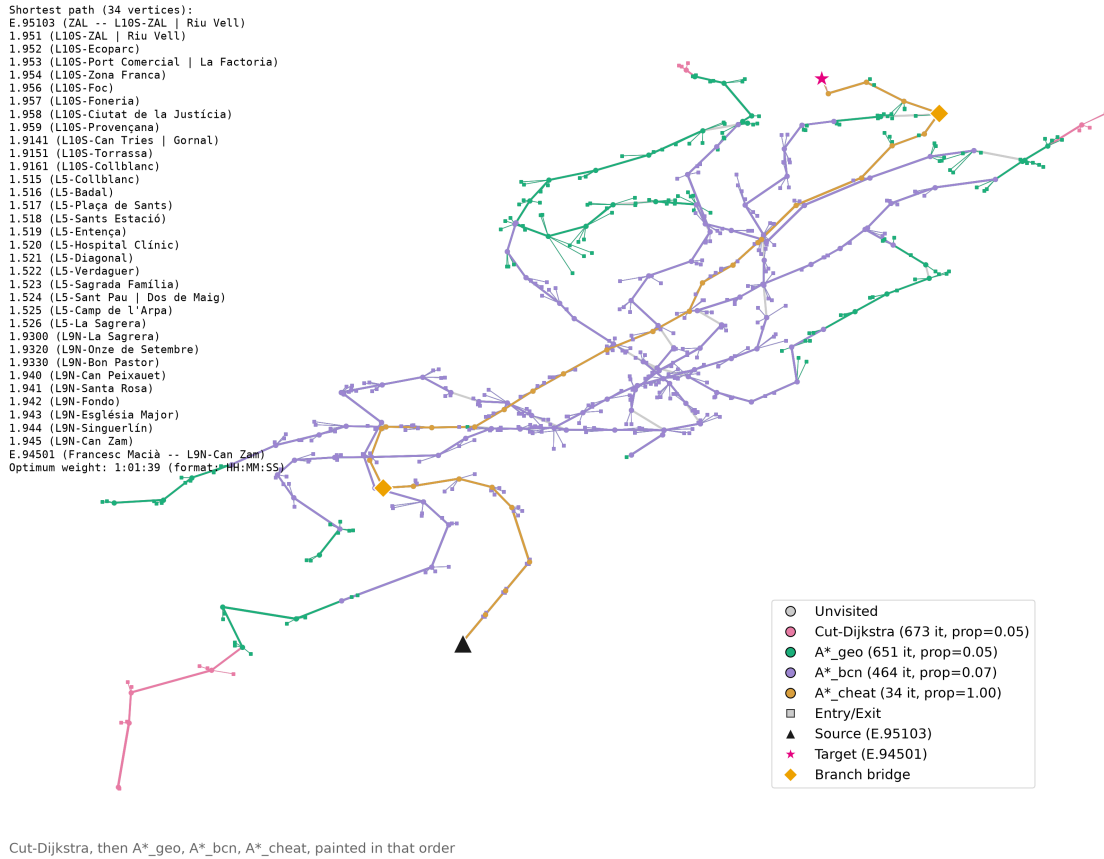


Figura 13: Exemple cas DB de nodes explorats pel Dijkstra, l'A*_geo, l'A*_bcn i l'A*_cheat, amb origen entrada ZAL de L10S – ZAL | Riu Vell i destí entrada Francesc Macià de L9N – Can Zam. Figura generada amb extracted_nodes_graph.py.

CB case (with the full graph): extracted nodes from Badajoz -- L1-Glòries to Residència sanitària -- L1-Hospital de Bellvitge

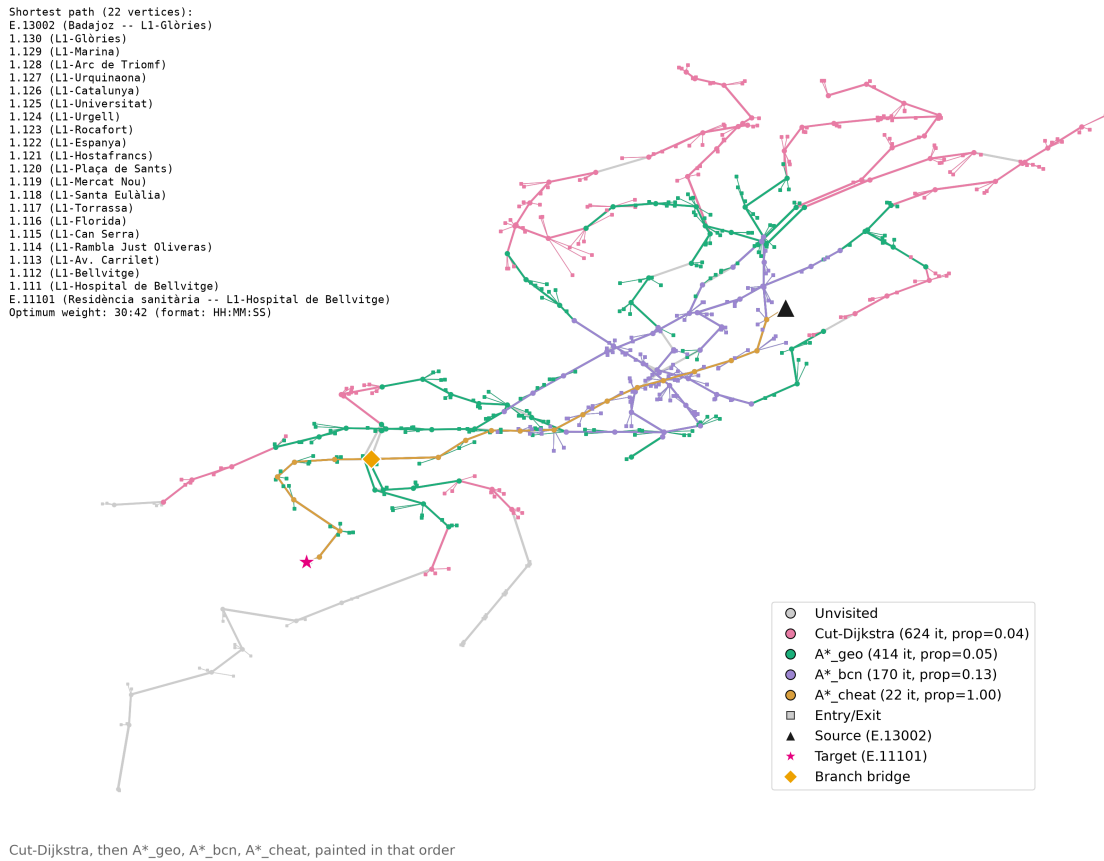
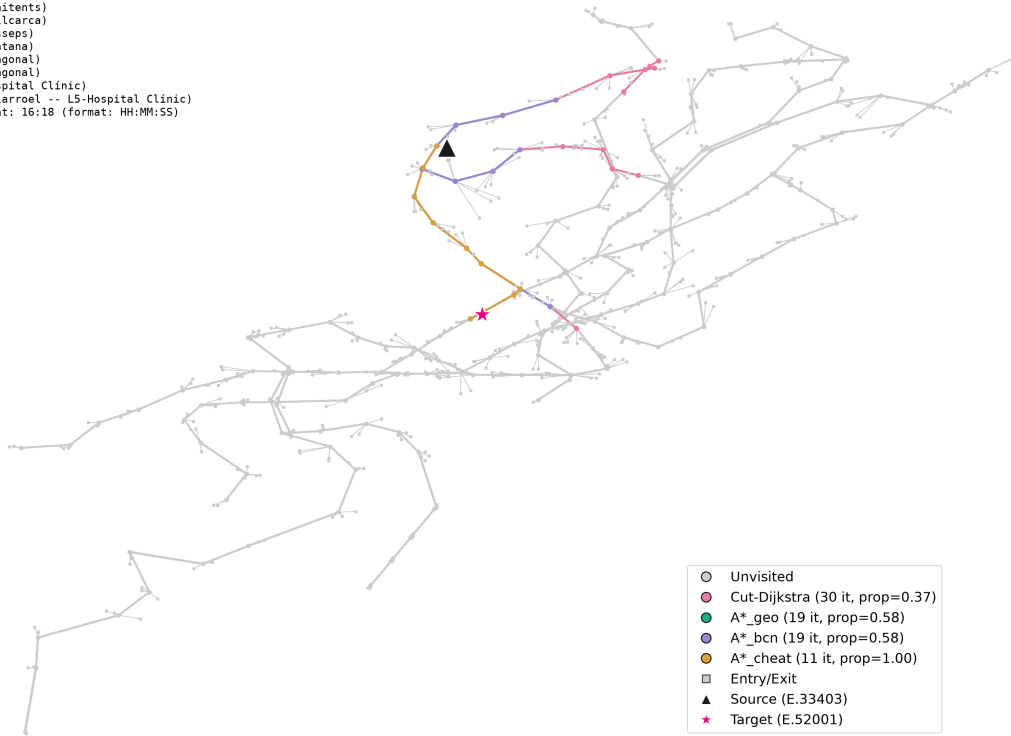


Figura 14: Mateix exemple del cas CB de la Figura 10 però incloent-hi A^*_{bcn} . L'origen és l'entrada Badajoz de L1 – Glòries i el destí l'entrada Residència sanitària de L1 – Hospital de Bellvitge. Figura generada amb `extracted_nodes_graph.py`.

CC case (without entrances in the graph): extracted nodes from Palau Olímpic Vall d'Hebron -- L3-Montbau to Villarroel -- L5-Hospital Clínic

Shortest path (11 vertices):
 E.33403 (Palau Olímpic Vall d'Hebron -- L3-Montbau)
 1.334 (L3-Montbau)
 1.333 (L3-Vall d'Hebron)
 1.332 (L3-Penitents)
 1.331 (L3-Vallcarca)
 1.330 (L3-Lesseps)
 1.329 (L3-Fontana)
 1.328 (L3-Diagonal)
 1.521 (L5-Diagonal)
 1.520 (L5-Hospital Clínic)
 E.52001 (Villarroel -- L5-Hospital Clínic)
 Optimum weight: 16:18 (format: HH:MM:SS)



Cut-Dijkstra, then A*_geo, A*_bcn, A*_cheat, painted in that order

Figura 15: Mateix exemple del cas CC de la Figura 11 però canviant el graf de treball pel reduït. L'origen és l'entrada Palau Olímpic Vall d'Hebron de L3 – Montbau i el destí l'entrada Villarroel de L5 – Hospital Clínic. Figura generada amb `extracted_nodes_graph.py`.

D Andanes i entrades del graf

Llistat 4: Sortida retallada del programa stops_report.py

```
$ python stops_report.py

=====
L1
=====
1.111 - Hospital de Bellvitge
    - E.1011101 - Ascensor - Residencia sanitaria
    - E.11101 - Residencia sanitaria

...

1.120 - Placa de Sants
    - E.12001 - Alcolea (escala mecanica) (only platform -->entry pathway)
    - E.12002 - Alcolea
    - E.12003 - Olzinelles
    - E.12011 - Cros
    - E.12012 - Riego

...

1.140 - Fondo
    - E.1014002 - Ascensor - Mossen Cinto Verdaguer
    - E.14001 - Sicilia
    - E.14002 - Mossen Cinto Verdaguer

=====
L2
=====
1.210 - Parallel
    - E.1021002 - Ascensor - Avinguda Parallel
    - E.21001 - Nou de la Rambla
    - E.21002 - Avinguda Parallel
    - E.21011 - Fontrodona
    - E.21012 - Ronda Sant Pau

...

1.227 - Badalona Pompeu Fabra
    - E.1022701 - Ascensor - Avinguda Mart Pujol
    - E.1022711 - Ascensor - La Creu
    - E.22701 - Avinguda Mart Pujol
    - E.22711 - La Creu
```

=====

L3

=====

1.314 - Zona Universitaria

- E.31401 - Avinguda Diagonal
- E.31402 - Facultat d'Estudis Empresarials
- E.31403 - Dr. Maranyon / Diagonal
- E.31404 - Parc Cervantes

...

1.325 - Liceu

- E.1032521 - Ascensor - La Rambla / Teatre Liceu
- E.1032531 - Ascensor - La Rambla / Teatre Liceu
- E.32501 - La Rambla / Mercat de la Boqueria
- E.32511 - La Rambla / Mercat de la Boqueria
- E.32521 - La Rambla / Teatre Liceu
- E.32531 - La Rambla / Teatre Liceu

...

1.339 - Trinitat Nova

- E.33901 - Palamos

=====

L4

=====

1.413 - La Pau

- E.1041301 - Ascensor - Rambla Guipuscoa / Ca n'Oliva
- E.41301 - Rambla Guipuscoa / Ca n'Oliva

...

1.434 - Trinitat Nova

- E.43401 - Pedrosa
- E.43402 - Aiguablava

=====

L5

=====

1.509 - Cornellà Centre

- E.1050901 - Ascensor - Placa Estacio
- E.50901 - Placa Estacio

...

- 1.534 - Vall d'Hebron
 - E.1053401 - Ascensor - Residencia Sanitaria
 - E.53401 - Residencia Sanitaria
 - E.53402 - Avinguda Jorda

=====

L9S

=====

- 1.901 - Aeroport T1
 - E.90101 - Aeroport T1

...

- 1.918 - Zona Universitaria
 - E.1091801 - Ascensor - Dr. Maranyon
 - E.91801 - Dr. Maranyon

=====

L9N

=====

- 1.930 - La Sagrera

...

- 1.945 - Can Zam
 - E.1094501 - Ascensor - Francesc Macia
 - E.94501 - Francesc Macia

=====

L10S

=====

- 1.951 - ZAL | Riu Vell
 - E.95101 - Carrer A / Carrer 5
 - E.95102 - Riu Llobregat
 - E.95103 - ZAL

...

- 1.916 - Collblanc
 - E.1091601 - Ascensor - Av. Sant Ramon
 - E.91601 - Av. Sant Ramon

=====

L10N

=====

1.930 - La Sagrera

...

1.936 - Gorg

- E.1093601 - Ascensor - Antoni Bori
- E.93601 - Antoni Bori

=====

L11

=====

1.1136 - Trinitat Nova

...

1.1140 - Can Cuias

- E.1114001 - Ascensor - Circumvallacio
- E.114001 - Circumvallacio
- E.114011 - Les Fustes

=====

FM

=====

1.9901 - Parallel

1.9902 - Parc de Montjuic

- E.990201 - Avinguda Miramar